# TEMA 4

# AWS IoT



# Contenidos

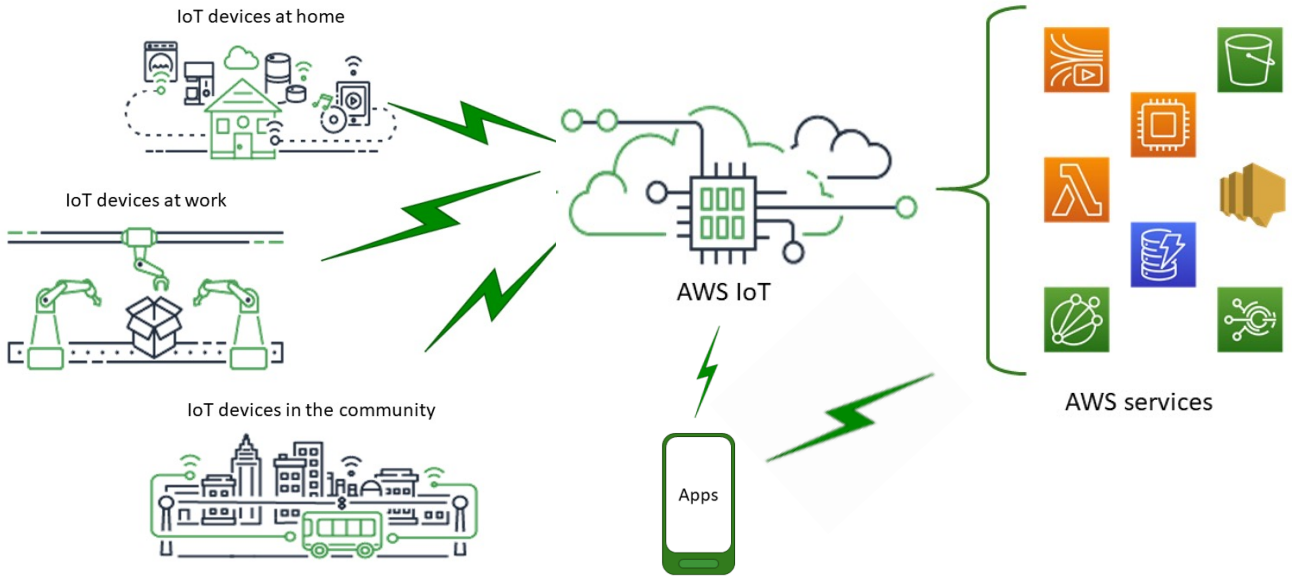
- Introducción
- Software de dispositivo
- Servicios de control
- Servicios de análisis de datos



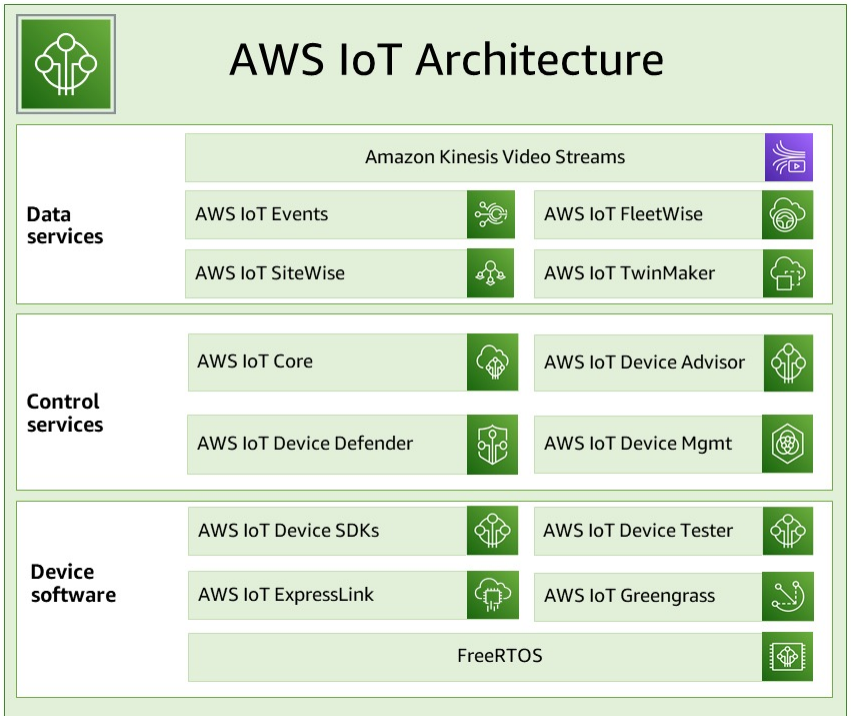
# Contenidos

- **Introducción**
- Software de dispositivo
- Servicios de control
- Servicios de análisis de datos

- AWS IoT
  - Plataforma IoT de AWS que proporciona
    - Software para integrar dispositivos IoT y Apps en soluciones basadas en AWS IoT
    - Servicios en la nube para conectar dispositivos IoT entre sí y con otros servicios de AWS



- AWS IoT
  - Los servicios de AWS IoT se organizan en tres categorías
    - Software de dispositivo
    - Servicios de control
    - Servicios de análisis de datos





# Contenidos

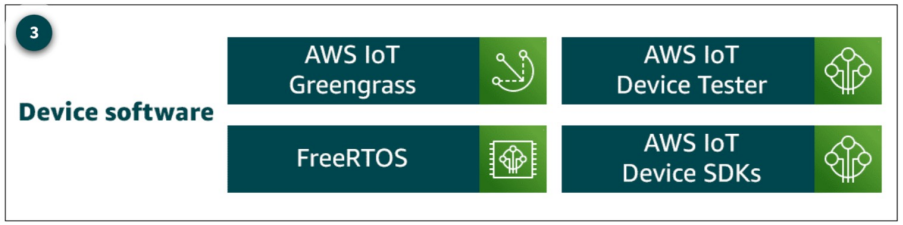
- Introducción
- **Software de dispositivo**
  - FreeRTOS
  - Device SDK
  - Device Client
  - Greengrass
- Servicios de control
- Servicios de análisis de datos



# Software de dispositivo

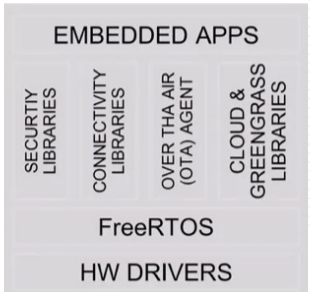


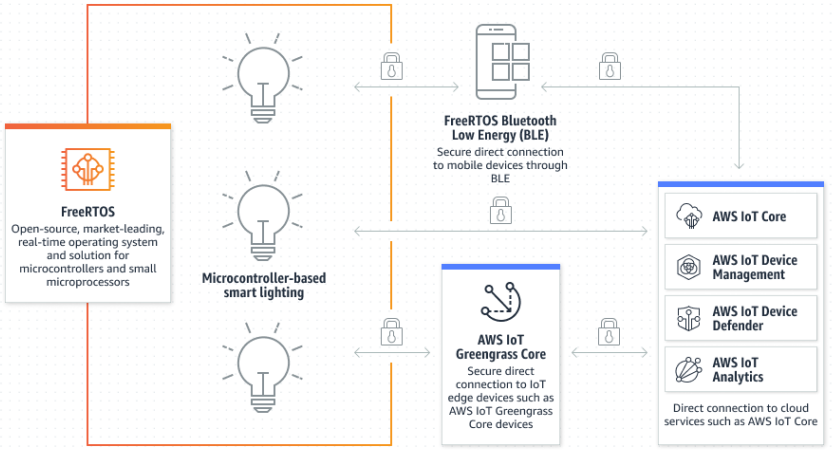
- AWS IoT tiene una oferta variada de software para dar soporte a distintos tipos de dispositivos
  - **FreeRTOS**
    - Sistema operativo de código abierto y en tiempo real para microcontroladores
    - Pensado para dispositivos pequeños y de baja potencia
  - **SDK de dispositivo**
    - Librerías de código abierto para diferentes lenguajes
    - Para desarrollar soluciones IoT en una variedad de plataformas HW
  - **Greengrass**
    - Combinación de plataforma de ejecución de código abierto y servicio en la nube para desarrollar, implantar y gestionar componentes software en dispositivos IoT
    - Para pasarelas de borde (*edge gateways*) con mayores capacidades de procesamiento y memoria
  - **Device Tester**
    - Herramienta de automatización de pruebas para dispositivos IoT que usen FreeRTOS o Greengrass



# Software de dispositivo

- FreeRTOS
  - Sistema operativo en tiempo real de código abierto y gratuito para microcontroladores y pequeños microprocesadores
    - Soporta más de 40 arquitecturas diferentes
    - Dispone de librerías para AWS
      - Conectividad local (WiFi) segura con pasarelas IoT de Greengrass
      - Conectividad segura con el AWS IoT Core incluyendo soporte para
        - Mensajería MQTT, sombras de dispositivo
        - Aprovisionamiento de dispositivos, trabajos remotos y actualización de *firmware* OTA (*Over-The-Air*)





# Software de dispositivo



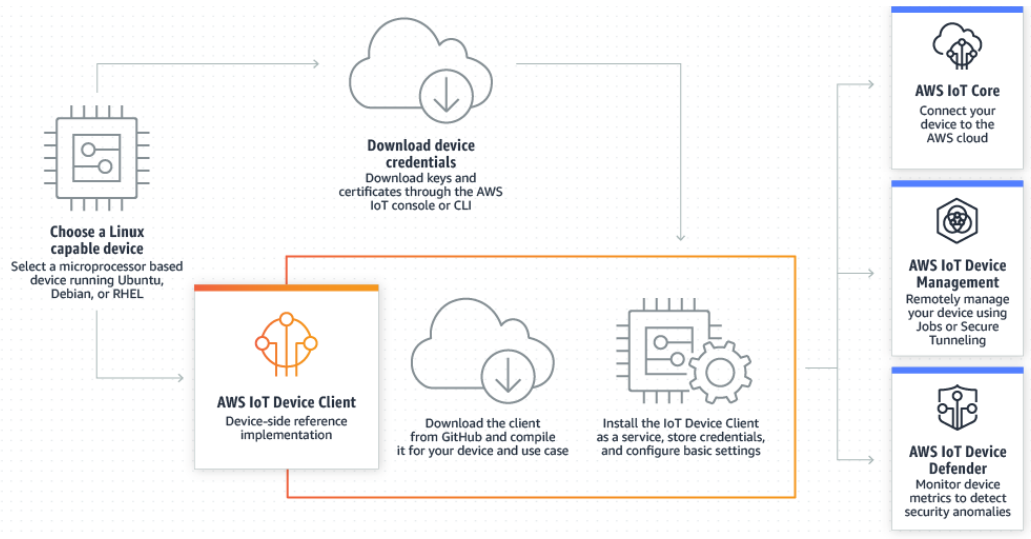
- Device SDK
  - Librerías de código abierto para la conexión segura de dispositivos a IoT Core
    - Protocolos: MQTT, HTTPS ou MQTT sobre WebSockets
    - Lenguajes: C++, Python, JavaScript, Java, Swift
- Device SDK for Embedded C
  - Colección de ficheros C para aplicaciones embebidas en dispositivos con recursos limitados
  - Incluye un cliente MQTT, un *parser* JSON y soporte a características de AWS IoT Core (sombras de dispositivo), de AWS IoT Device Management (trabajos remotos, aprovisionamiento) y AWS IoT Device Defender
- Mobile SDKs
  - Librerías para aplicaciones móviles que usen las API de AWS, incluídas las de AWS IoT Core
  - Soporta comunicación con dispositivos IoT mediante MQTT
    - Plataformas: Android, iOS



# Software de dispositivo



- Device client
  - Implementación de referencia de código abierto en C++ de un cliente de AWS IoT Core
  - Proporciona un cliente básico para dispositivos IoT basados en Linux con varias funcionalidades ya implementadas
    - Soporte de características de AWS IoT Core: conectividad segura, sombras de dispositivo
    - Un cliente MQTT
    - Soporte de características de AWS IoT Device Management: conexión remota al dispositivo a través de un túnel SSH seguro, trabajos remotos y aprovisionamiento
    - Soporte a la monitorización de métricas de seguridad de AWS IoT Device Defender

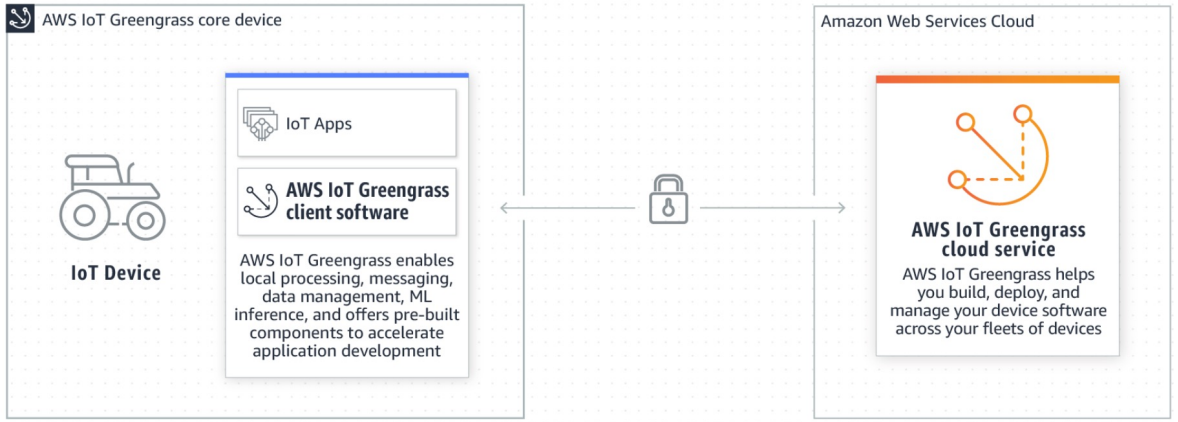




# Software de dispositivo

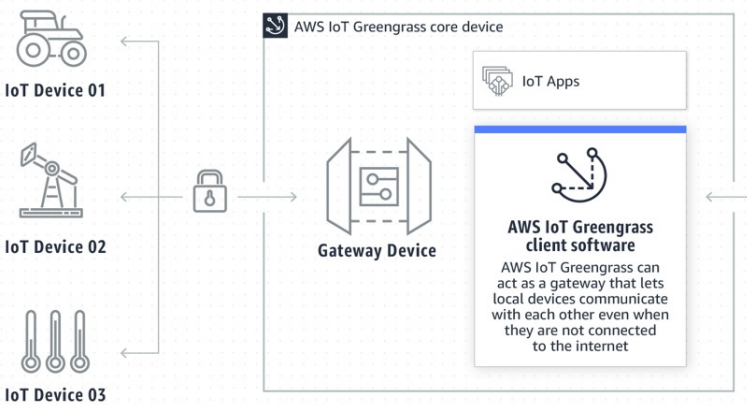


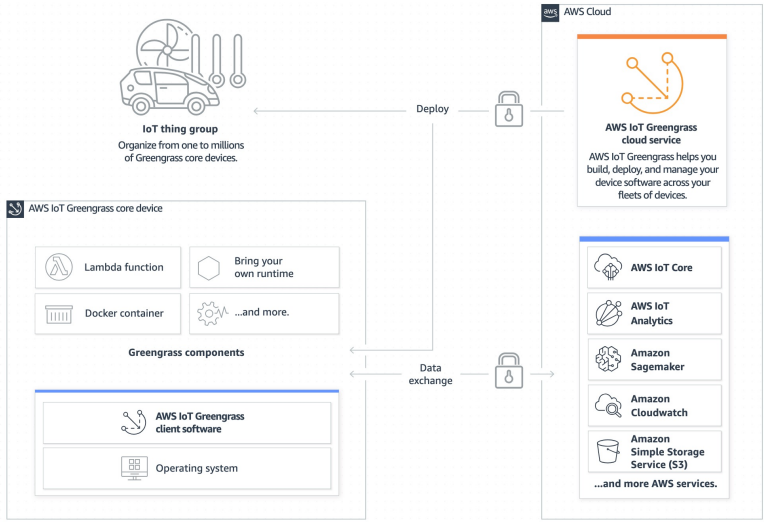
- Greengrass
  - Servicio AWS de computación en el borde para IoT que proporciona gestión de dispositivos de borde y del ciclo de vida de las aplicaciones ejecutadas en ellos
  - Formado por 2 subsistemas
    - **Dispositivo núcleo (*core device*) de Greengrass**
      - Dispositivos de borde que ejecutan el código cliente de Greengrass y actúan como pasarelas IoT
      - El cliente Greengrass está disponible para Windows y diferentes distribuciones de Linux, como Ubuntu o Raspberry Pi OS, sobre dispositivos con arquitecturas ARM o x86
      - Soporta el despliegue de aplicaciones que usen procesos nativos del SO, funciones Lambda, contenedores Docker, o cualquier otro entorno personalizado de ejecución
    - **El servicio Greengrass en la nube**
      - Se encarga de la gestión de los dispositivos *core* y del ciclo de vida de sus aplicaciones



# Software de dispositivo

- Greengrass
  - **Principales funcionalidades**
    - Actúa como pasarela de borde (*edge gateway*) proporcionando a otros dispositivos IoT
      - Comunicaciones seguras y mensajería MQTT entre sí y con IoT Core
      - Análisis de datos y reacción a eventos cercana a los procesos que los generan
      - Filtrado y agregación de datos antes de subirlos a la nube
      - Conexión a servicios de AWS o de terceros
    - Ejecuta aplicaciones en el borde usando los datos recogidos localmente
      - **Ejemplo:** predicciones basadas en modelos de aprendizaje automático
    - Proporciona componentes SW modulares que extienden la funcionalidad de los dispositivos
    - Gestiona los dispositivos de borde como cosas y grupos de cosas en la nube





# Software de dispositivo



- Greengrass
  - **Greengrass Core Software**
    - Software instalado en un dispositivo *core* de Greengrass y compuesto de
      - **Núcleo**
        - Parte que implementa la funcionalidad mínima de Greengrass en el dispositivo
        - Gestiona el despliegue, orquestación y el ciclo de vida de otros componentes y facilita la comunicación entre ellos
      - **Componentes opcionales**
        - Módulos SW que se despliegan y ejecutan en los dispositivos
        - Añaden funciones como *streaming* de datos y vídeo, inferencia ML, sombras de dispositivo, etc
    - Principales funcionalidades que implementa Greengrass Core
      - Conexión segura con AWS con autenticación y autorización de dispositivos
      - Agente de mensajes MQTT para dispositivos IoT clientes
      - Mensajería MQTT entre componentes y con IoT Core
      - Mensajería local (IPC) entre componentes
      - Gestión del ciclo de vida de los componentes
      - Actualizaciones OTA seguras del SW de Greengrass y de los componentes
      - Gestión de sombras de dispositivo para componentes y dispositivos IoT clientes
      - Procesamiento de *streams* de datos y exportación a AWS (Kinesis, S3, IoT SiteWise, ... )
      - Despliegue e invocación de componentes y funciones Lambda



# Software de dispositivo



- Greengrass
  - **Componentes**
    - Todo el SW desplegado con Greengrass se modela como un componente
      - Pueden representar *scripts*, aplicaciones, instaladores de entornos de ejecución (*runtimes*), librerías o cualquier código que pueda ejecutar el dispositivo
      - Pueden ser proporcionados por AWS, propios o de un tercero
      - Pueden tener dependencias entre ellos
        - Ejemplo: los componentes que ejecutan aplicaciones Python pueden depender de un componente que lo instale
    - Un componente está compuesto por los siguientes recursos
      - **Receta**
        - Un fichero JSON o YAML que describe los detalles, configuración y parámetros del componente
      - **Artefacto**
        - El código fuente, binarios o *scripts* del SW que se ejecutará en el dispositivo
        - Puede ser también una función Lambda, un contenedor Docker o un entorno de ejecución personalizado
      - **Dependencias**
        - Relaciones con otros componentes que permiten hacer actualizaciones automáticas o reiniciar componentes dependientes, p.ej. la actualización de un componente de cifrado provoca la actualización y reinicio automáticos de un componente de mensajería seguro que dependa de él



# Software de dispositivo



- Greengrass
  - **Receta de un componente**
    - Fichero que define los detalles, dependencias, artefactos y ciclos de vida de un componente
      - Formatos: JSON o YAML
      - El ciclo de vida especifica los comandos para p.ej. instalar, iniciar o apagar el componente
      - Soporte multiplataforma: manifiestos con definiciones específicas para cada plataforma soportada
    - Cuando se despliega un componente Greengrass usa la receta para comprobar si es compatible con el dispositivo e identificar las dependencias y artefactos a desplegar y los comandos a ejecutar para instalarlo e iniciarlo

```
{
  "RecipeFormatVersion": "2020-01-25",
  "ComponentName": "com.example.HelloWorld",
  "ComponentVersion": "1.0.0",
  "ComponentDescription": "My first AWS IoT Greengrass component.",
  "ComponentPublisher": "Amazon",
  "ComponentConfiguration": {
    "DefaultConfiguration": {
      "Message": "world"
    }
  },
  "Manifests": [
    {
      "Platform": {
        "os": "linux"
      },
      "Lifecycle": {
        "run": "python3 -u {artifacts:path}/hello_world.py {configuration:/Message}"
      }
    },
    {
      "Platform": {
        "os": "windows"
      },
      "Lifecycle": {
        "run": "py -3 -u {artifacts:path}/hello_world.py {configuration:/Message}"
      }
    }
  ]
}
```

Pueden usarse variables en las recetas para acceder a información de configuración del componente o del dispositivo

- Sintaxis: **{variable}**
- Ejemplos:

**iot:thingName**

El nombre de la cosa asociada al dispositivo en AWS IoT

**component_dependency_name:artifacts:decompressedPath**

La ruta raíz en la que se descomprimieron los artefactos del componente

**component_dependency_name:work:path**

La ruta de trabajo del componente



# Software de dispositivo



- Greengrass
  - **Receta de un componente**
    - Componente *TokenExchangeService*
      - Componente de Greengrass para ser definido como dependencia en la receta de aquellos componentes que necesiten credenciales temporales para acceder a servicios AWS
      - Cuando un componente con esta dependencia crea un cliente con el AWS SDK, se conecta a un servidor local en el dispositivo que obtiene del proveedor de credenciales de AWS IoT las credencias de acceso temporales a los servicios AWS
      - Las credenciales se almacenan en la memoria del dispositivo y el tiempo de caducidad es de una hora por defecto

```
ComponentDependencies:
  aws.greengrass.TokenExchangeService:
    VersionRequirement: '^2.0.0'
    DependencyType: HARD
```



# Software de dispositivo



- Greengrass
  - **Despliegues**
    - Definen los componentes y configuraciones que se envían a los dispositivos
      - Cada dispositivo solo puede tener un despliegue a la vez
      - Se despliegan usando trabajos remotos tanto en dispositivos individuales como en grupos de dispositivos
      - Proporciona opciones para controlar que dispositivos reciben una actualización y como se despliega
    - Cuando un dispositivo *core* recibe un despliegue
      - Comprueba si es compatible con la plataforma del dispositivo
      - Comprueba si hay incompatibilidad entre las versiones de las dependencias de los componentes
      - Notifica a cada componente presente en el dispositivo para darle la oportunidad de retrasar el despliegue, p.ej. porque el dispositivo tenga un nivel de batería bajo o esté ejecutando un proceso que no pueda ser interrumpido
    - Los despliegues son **continuos**
      - Cuando se crean, se despliegan en los dispositivos destino que estén en línea
      - Cuando se añade un nuevo dispositivo destino o vuelve a estar en línea uno que no lo estaba, se le envía la versión más actual del despliegue



# Software de dispositivo



- Greengrass
  - **Componentes de aprendizaje máquina** (*ML, Machine Learning*)
    - Componentes para realizar inferencias en el borde usando modelos ML entrenados en la nube
    - Categorías de componentes ML
      - Modelo
        - Componente que contiene el modelo ML como artefacto de Greengrass
      - Entorno de ejecución (*runtime*)
        - Componente que contiene el *script* que instala el entorno de ejecución de modelos ML (*ML framework*) y sus dependencias
      - Inferencia
        - Componente que contiene el código de inferencia
        - Define dependencias con los otros componentes para instalar el entorno de ejecución y descargar el modelo ML
    - Greengrass proporciona ejemplos de componentes de inferencia y modelos para los siguientes entornos de ejecución
      - Deep Learning Runtime (DLR)
      - TensorFlow Lite
    - Pueden crearse componentes personalizados en cualquiera de las categorías y combinarse entre ellos o con los proporcionados por AWS



# Contenidos

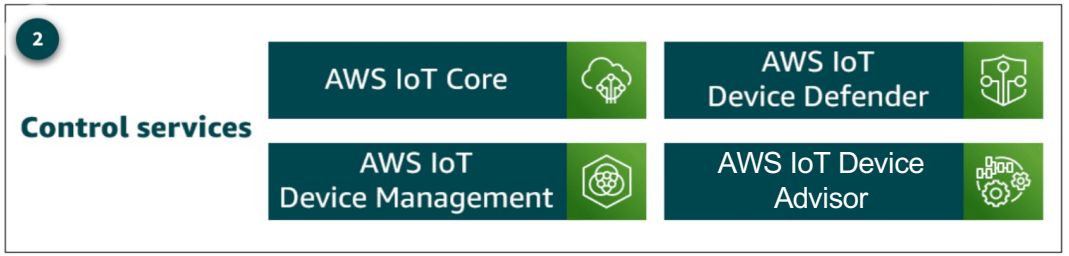
- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - Device Management
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



# Servicios de control



- Son los servicios que forman el núcleo de la funcionalidad de AWS IoT
  - **IoT Core**
    - Servicio para la conexión y comunicación segura de dispositivos IoT con aplicaciones en la nube y otros dispositivos
  - **Device Management**
    - Servicio para el registro, organización, monitorización y gestión remota de flotas de dispositivos IoT a gran escala
  - **Device Defender**
    - Servicio para la auditoría de configuraciones de seguridad en flotas de dispositivos IoT
  - **Device Advisor**
    - Servicio para la validación automática de dispositivos IoT





# Contenidos

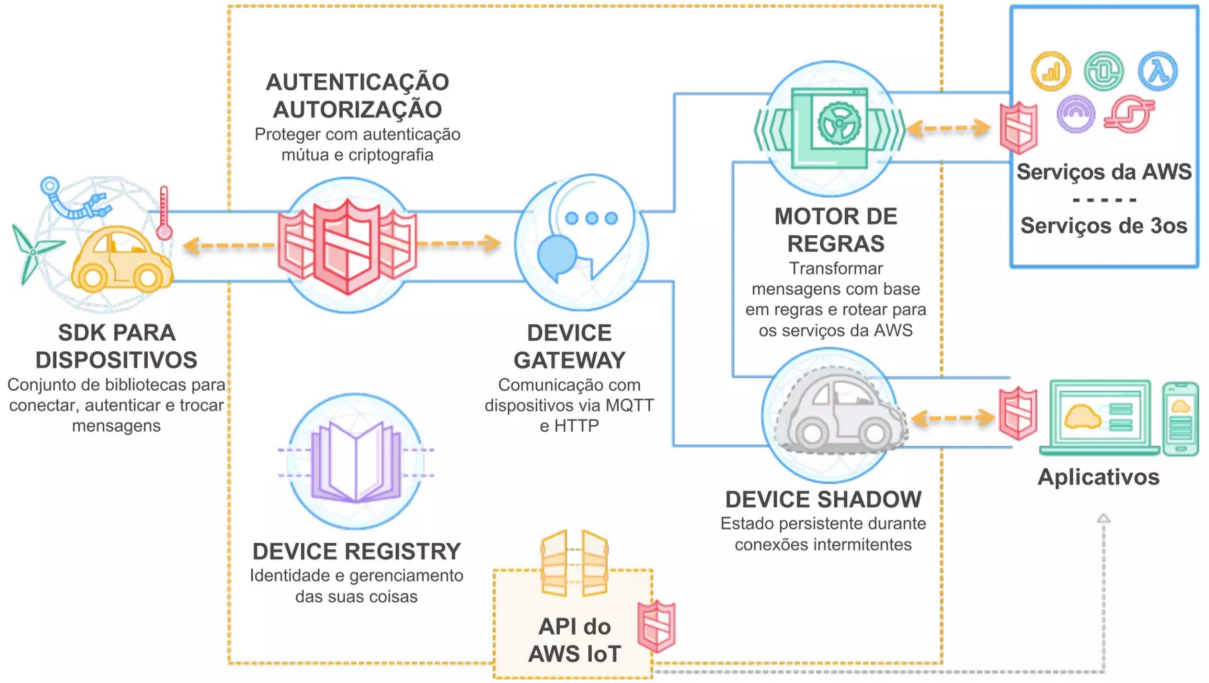
- Introducción
- Software de dispositivo
- **Servicios de control**
  - **IoT Core**
    - **Introducción**
    - Seguridad
    - Registro de dispositivos
    - Motor de reglas
    - Sombras de dispositivo
  - Device Management
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



# IoT Core



- Introducción
  - Proporciona servicios para
    - Conectar de forma segura dispositivos IoT a la nube de AWS
    - Registrar dispositivos
    - Intercambiar mensajes entre dispositivos y/o con servicios AWS y aplicaciones
    - Gestionar el estado persistente de los dispositivos (sombra)

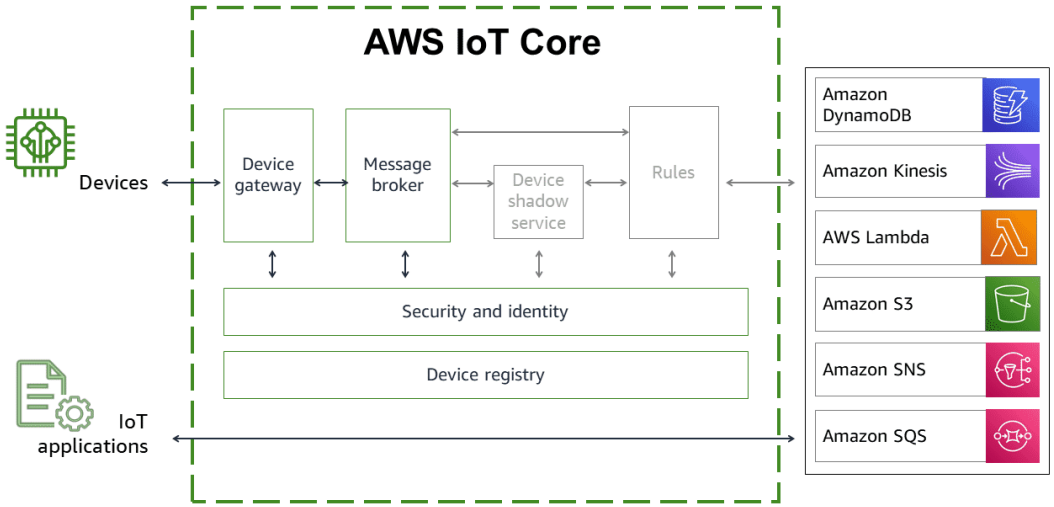




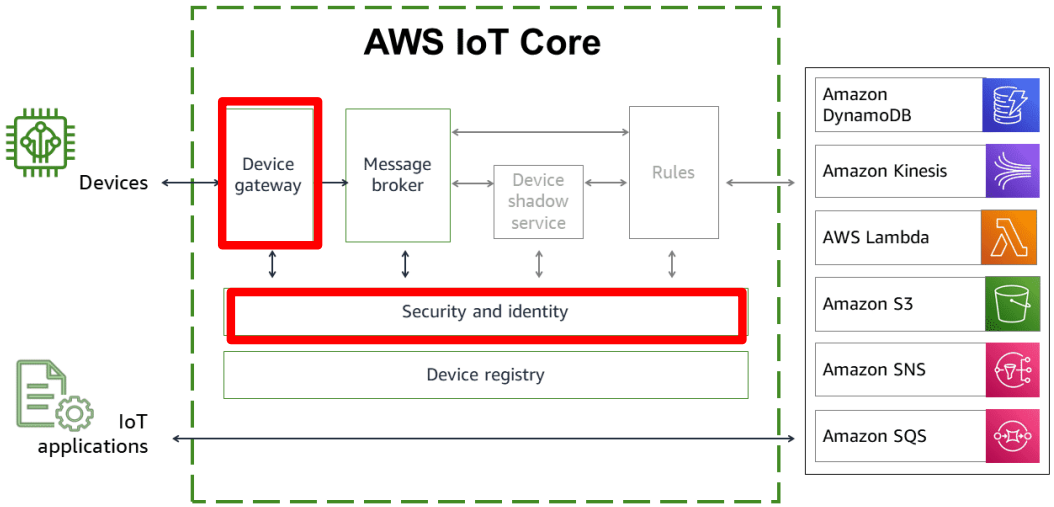
# IoT Core



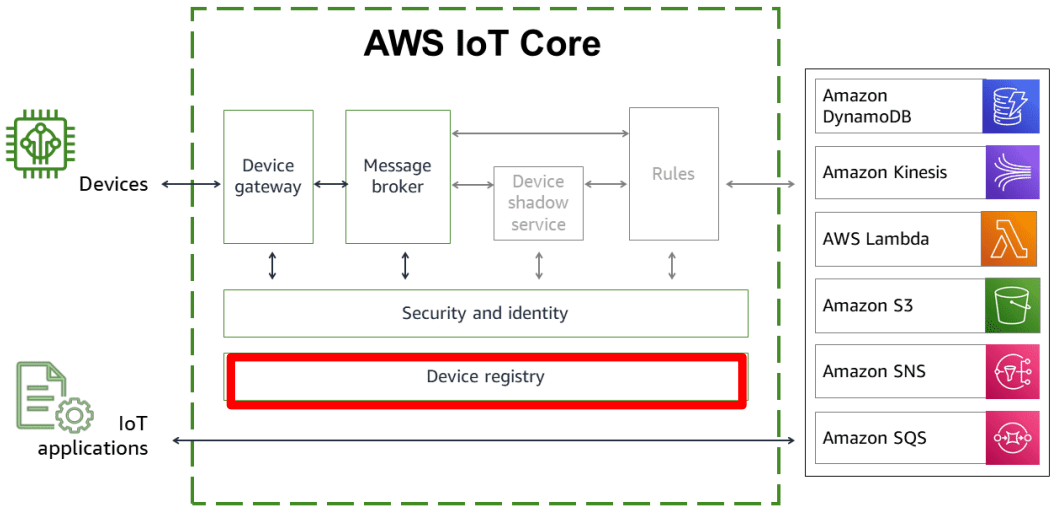
- Introducción
  - Los servicios se agrupan en tres categorías
    - **Servicios de control**
      - Proporcionan recursos de seguridad, gestión y registro de dispositivos
    - **Servicios de mensajes**
      - Proporcionan comunicación segura y gestión de los mensajes entre los dispositivos IoT e IoT Core
    - **Servicios de datos**
      - Proporcionan persistencia y gestión del estado de los dispositivos incluso cuando no tengan conexión



- Servicios de control
  - **Seguridad e identidad**
    - Proporciona los mecanismos de seguridad requeridos por los dispositivos y aplicaciones para autenticarse y comunicarse de manera segura con IoT Core
      - Certificados, autenticación mutua, cifrado, políticas de autorización
  - **Pasarela de dispositivos** (*Device Gateway*)
    - Sirve como punto de acceso seguro de los dispositivos a los servicios de AWS IoT
    - Es responsable de la autenticación y autorización de dispositivos y aplicaciones conectados a IoT Core mediante los mecanismos de seguridad de AWS



- Servicios de control
  - **Registro de dispositivos** (*Device Registry*)
    - Base de datos de dispositivos
    - Almacena metadatos estáticos y atributos de los dispositivos
      - Números de serie, fabricante, versión de *firmware*, capacidades del dispositivo, etc

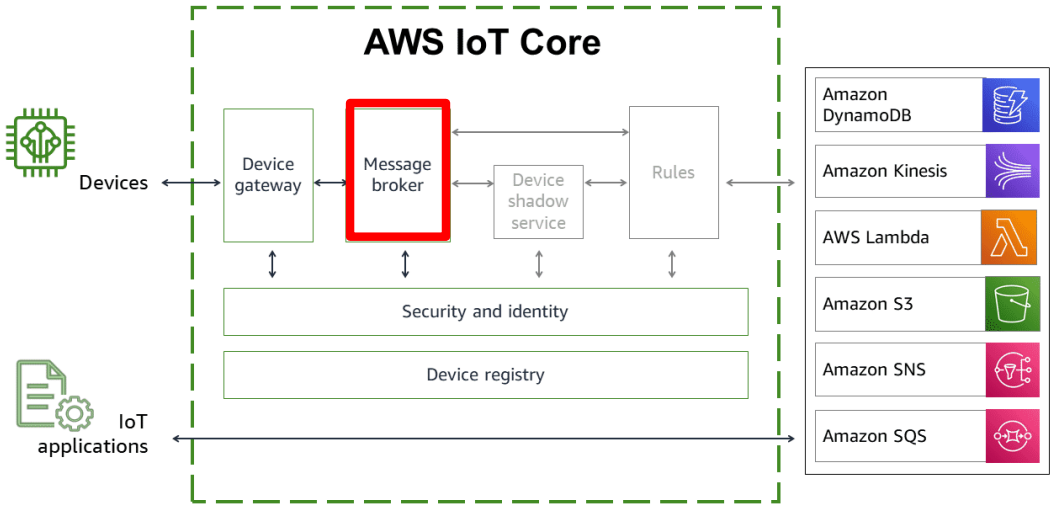




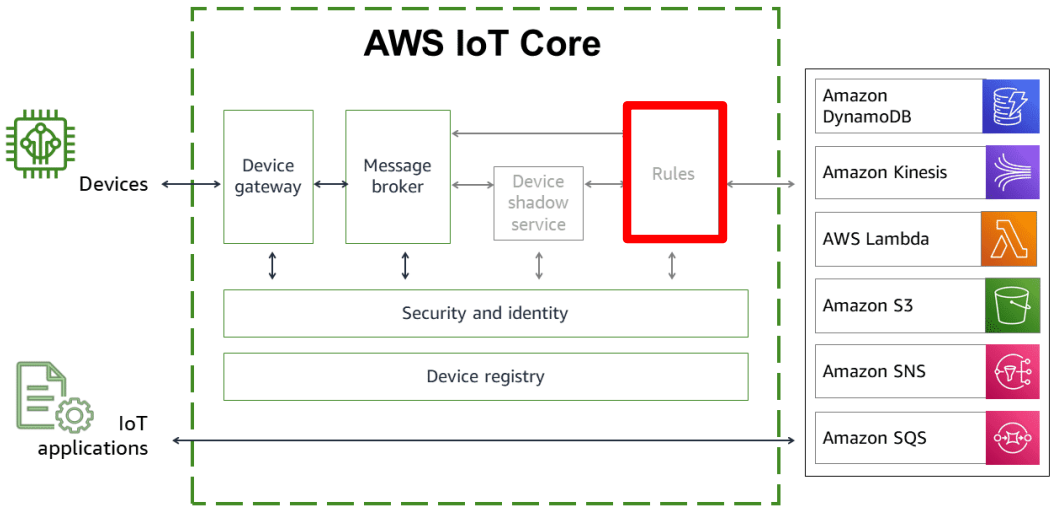
# IoT Core



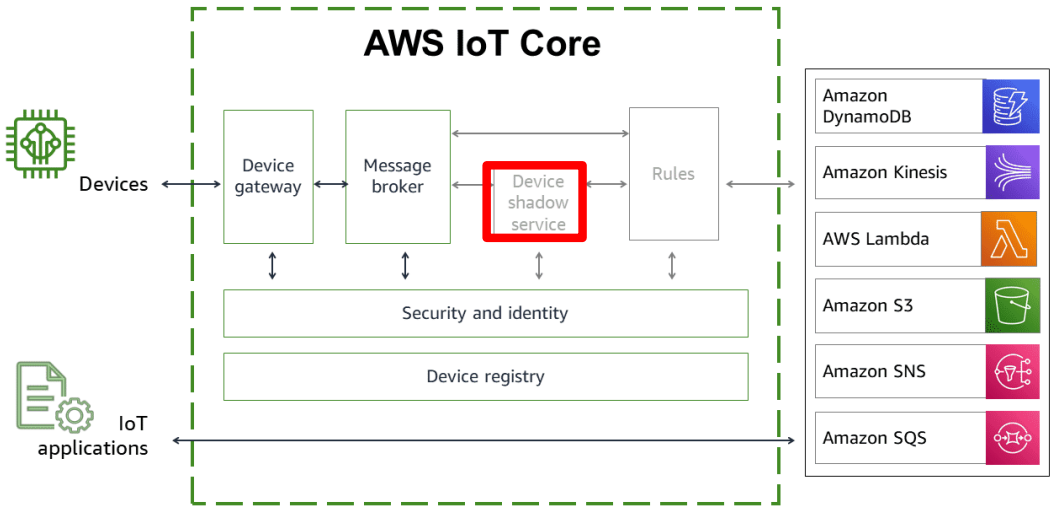
- Servicios de mensajes
  - **Agente de mensajes** (*Message Broker*)
    - Dispositivos y aplicaciones IoT se comunican entre sí y con los servicios AWS usando MQTT
    - Agente de alto rendimiento con baja latencia que intercambia mensajes de forma segura entre dispositivos y aplicaciones IoT usando un modelo de publicación/suscripción
      - Los productores envían datos publicando un mensaje en un tema
      - Los consumidores reciben datos suscribiéndose a temas
      - Los temas filtran y seleccionan los mensajes
    - Protocolos que soporta
      - MQTT, MQTT sobre WSS (*Websockets Secure*) y HTTPS



- Servicios de mensajes
  - **Motor de reglas** (*Rules Engine*)
    - Aplica reglas de transformación a los datos del agente de mensajes y los envía la otros servicios AWS
    - Usa un lenguaje basado en SQL para seleccionar los datos en los mensajes y transformarlos



- Servicios de datos
  - **Sombras de Dispositivo**
    - Mantiene el estado (sombra) de un dispositivo para que las aplicaciones puedan comunicarse con él aunque no esté en línea





# Contenidos

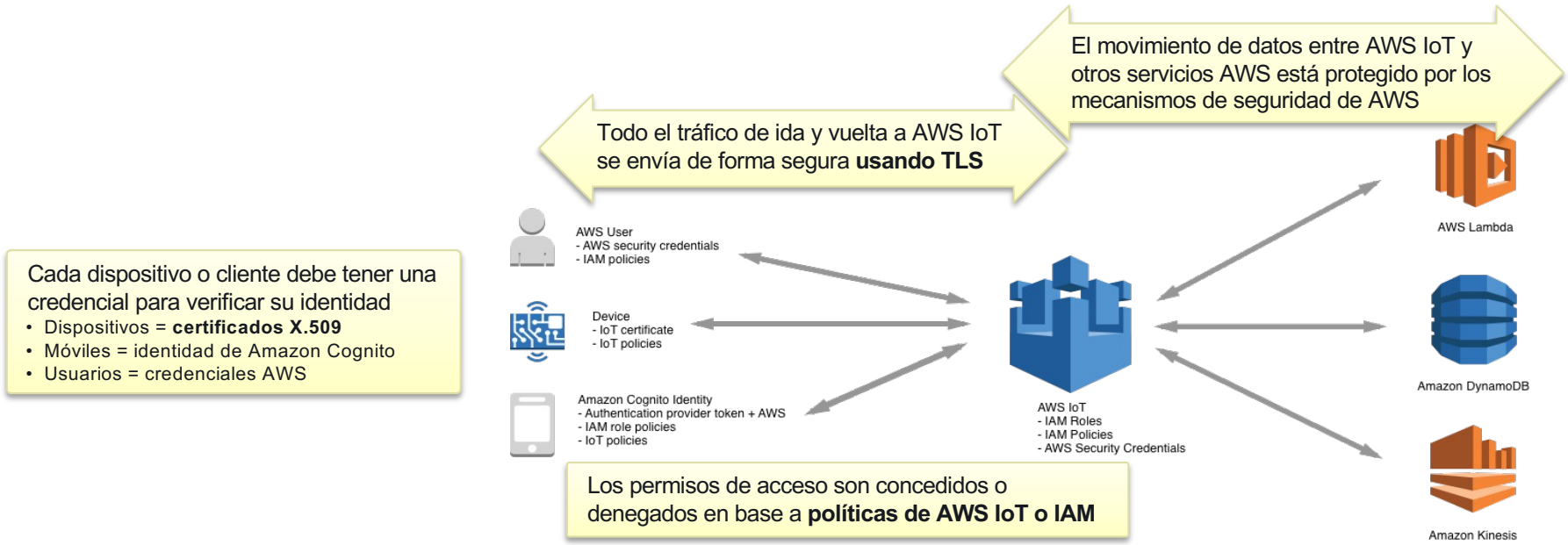
- Introducción
- Software de dispositivo
- **Servicios de control**
  - **IoT Core**
    - Introducción
    - **Seguridad**
    - Registro de dispositivos
    - Motor de reglas
    - Sombras de dispositivo
  - Device Management
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



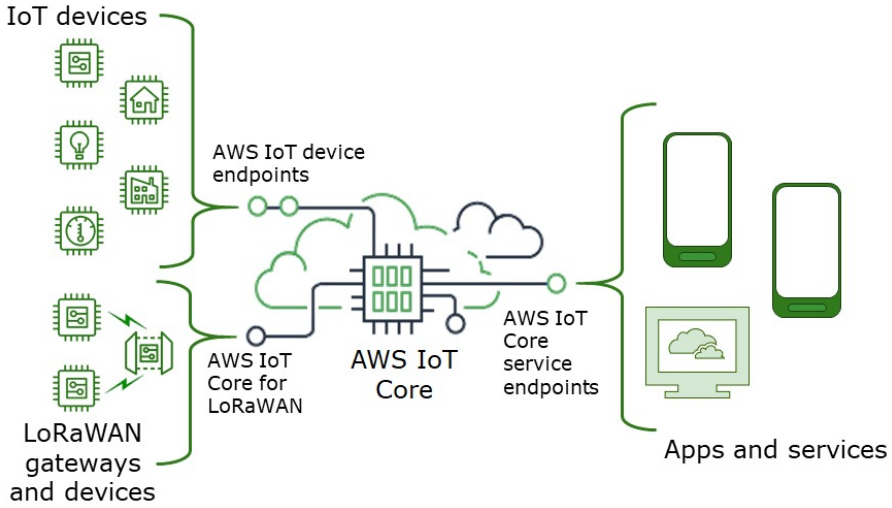
# Seguridad



- Visión general de la seguridad en AWS IoT
  - La seguridad en un sistema IoT tiene tres componentes fundamentales
    - Autenticación: verifica la identidad de un usuario, dispositivo, aplicación o servicio
    - Autorización: concede o niega los permisos de acceso a un recurso
    - Comunicaciones seguras
  - En un sistema IoT los dispositivos, pasarelas, agentes de mensajes, usuarios y aplicaciones tienen que autenticarse y estar autorizados para realizar sus funciones
  - La autenticación y autorización puede realizarlas AWS IoT o puede configurarse un autorizador personalizado que use métodos de autenticación y autorización propios



- Conexión a IoT Core
  - IoT Core soporta conexiones de dispositivos IoT, pasarelas LoRaWan, servicios y aplicaciones
    - Los dispositivos se conectan a IoT Core para enviar y recibir datos
    - Las aplicaciones y servicios para controlar y gestionar los dispositivos IoT y procesar los datos
  - Los *endpoints* y métodos de autenticación y autorización son diferentes




¡¡Recuerda...!!

**Responsabilidad compartida**

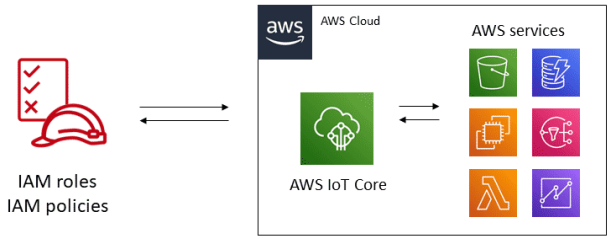
Es responsabilidad del usuario mantener seguras las credenciales de IoT Core en los dispositivos

# Seguridad

- Autenticación y autorización
  - Dispositivos
    - Certificados X.509
    - Políticas AWS IoT
  - Usuarios y aplicaciones
    - Identidades IAM
    - Roles y políticas IAM
  - Móviles (pueden actuar como ambos)
    - Identidades de Amazon Cognito
    - Políticas AWS IoT
    - Roles y políticas IAM




Una visión general sobre como los dispositivos se conectan de forma segura a IoT Core y como este interactúa con otros servicios usando un rol IAM con los permisos asignados por políticas IAM.




Las políticas de seguridad de los usuarios y aplicaciones se definen usando IAM. Los usuarios pueden asumir roles, que confieren los permisos necesarios para tareas específicas




**Principio del privilegio mínimo**

Configurar los permisos limitados a las necesidades que sean necesarias

P.ej. un ingeniero de sistemas puede necesitar activar o desactivar sensores, mientras que un operador solo podrá leer datos de sensores y recoger registros de datos

- Conexión de dispositivos
  - Se usa el protocolo de **autenticación mutua** de TLS
    - La autenticación del cliente, en la que AWS IoT verifica la identidad del dispositivo
      - El dispositivo envía su certificado X.509 a IoT Core que lo valida contra un registro de certificados
      - Son válidos tanto certificados emitidos por AWS IoT como por otras CA (*Certificate Authorities*) siempre que el certificado se registre previamente en AWS IoT y no esté revocado o caducado



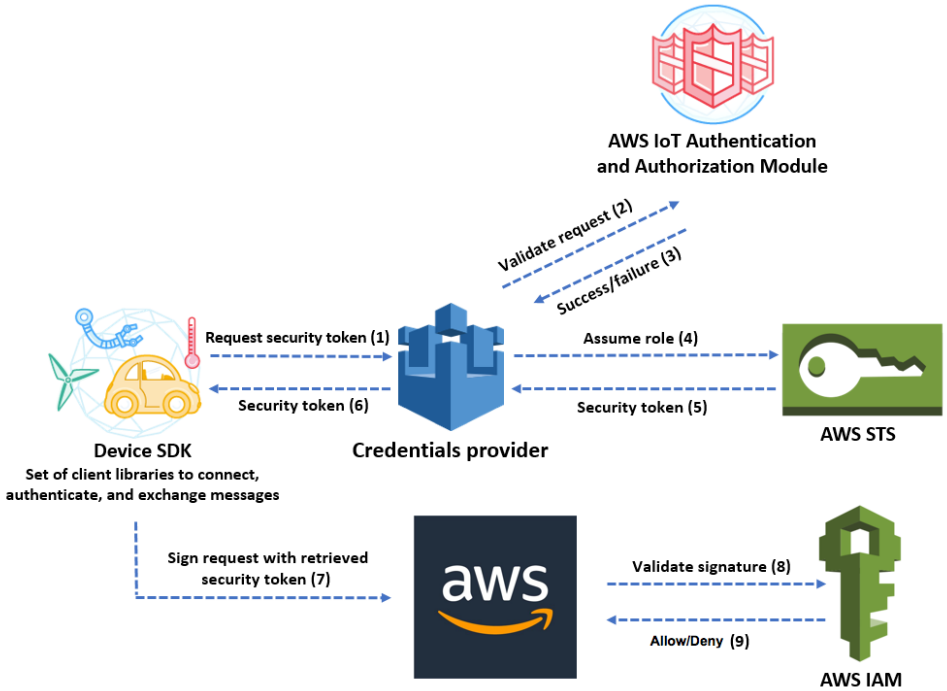

    - La autenticación del servidor, en la que el dispositivo verifica que está conectándose con un *endpoint* de AWS IoT
      - IoT Core envía su certificado X.509 al dispositivo que valida su cadena de certificados
      - Es el mismo método que se usa cuando se accede con un navegador web a una URL HTTPS




**Recomendación**

- Usar los *endpoints* ATS (*Amazon Trust Services*) de IoT Core
- Guardar en el dispositivo los certificados raíz de la CA de Amazon para los métodos de cifrado que se usen
- Planificar la rotación de los certificados raíz de la CA de Amazon antes de que caduquen

- Conexión de dispositivos
  - Para la conexión con servicios AWS que no admitan certificados y usen Signature Version 4 (SigV4) IoT Core tiene un **proveedor de credenciales** que permite que los dispositivos obtengan un *token* temporal válido mediante su certificado
    - Evita que los dispositivos tengan que almacenar las credenciales SigV4 localmente
    - Para la autorización se usa un rol IAM que es asumido por el proveedor de credenciales a través del servicio STS (*Security Token Service*). Los permisos asociados al rol son verificados usando IAM desde el servicio AWS invocado por el dispositivo





# Seguridad



- Autorización
  - La autorización es el proceso de concesión de permisos a una identidad autenticada
  - La identidad solo puede ejecutar operaciones de AWS IoT si tiene una política que le otorgue permiso
    - Dispositivos ← Políticas AWS IoT
    - Usuarios y aplicaciones ← Políticas IAM
  - **Políticas AWS IoT**
    - Son documentos JSON que siguen las mismas convenciones que las políticas IAM
    - Se anexan a los certificados X.509 de los dispositivos
    - Controlan el acceso del dispositivo al plano de datos de IoT Core, p.ej. conectarse al agente de mensajes, enviar o recibir mensajes MQTT, publicar en un tema MQTT específico, etc

Ejemplo de política AWS IoT que:
- Concede permiso para conectarse a IoT Core con un ID de cliente que coincida con el nombre de una cosa registrada
- Concede permiso para publicar en un tema MQTT con el mismo nombre que el ID de cliente

```
{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action":["iot:Publish"],
        "Resource": ["arn:aws:iot:us-east-1:123456789012:topic/${iot:Connection.Thing.ThingName}"]
      },
      {
        "Effect": "Allow",
        "Action": ["iot:Connect"],
        "Resource": ["arn:aws:iot:us-east-1:123456789012:client/${iot:Connection.Thing.ThingName}"]
      }
    ]
}
```



# Contenidos

- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
    - Introducción
    - Seguridad
    - **Registro de dispositivos**
    - Motor de reglas
    - Sombras de dispositivo
  - Device Management
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos

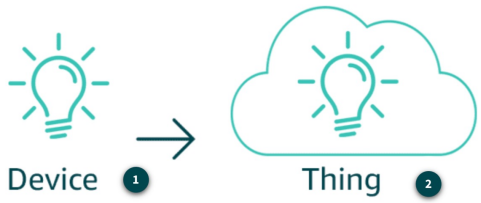


# Registro de dispositivos



- Cosas
  - Para conectar un dispositivo a IoT Core hay que aprovisionarlo con lo necesario para establecer una conexión segura
  - Para realizar el aprovisionamiento se crea y registra una cosa en IoT Core

Una **cosa** es la representación virtual en la nube de un dispositivo físico

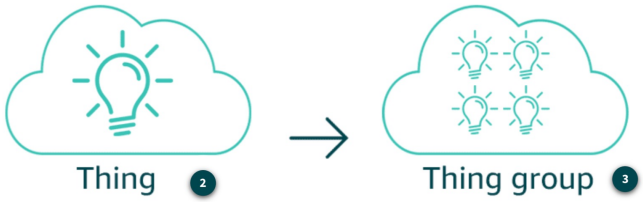



**Cosa**
- Nombre
- Atributos (opcional) ≤3
- Tipo (opcional) +50 atributos

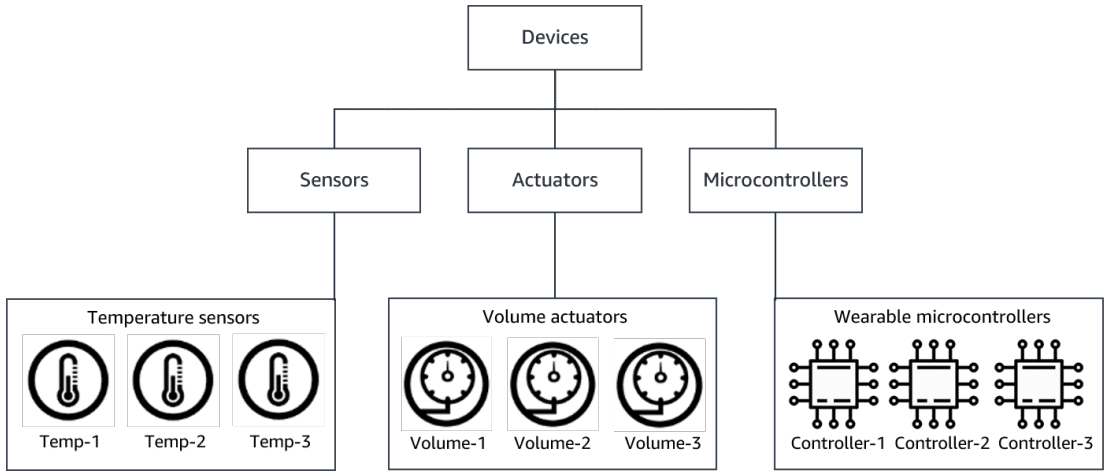
- **Nombre**
  - Tiene que ser único dentro de una región de AWS
- **Atributos**
  - Pares clave-valor que almacenan información como función, localización, especificaciones técnicas, etc.
  - Pueden crearse 3 atributos por cosa
  - En el registro de dispositivos se pueden hacer búsquedas por atributos y etiquetas
- **Tipo**
  - Organiza las cosas en categorías lógicas que comparten atributos comunes
  - Cada tipo puede tener hasta 50 atributos
  - Cada cosa solo puede tener un tipo y los nombres de tipos tienen que ser únicos en una cuenta de AWS

# Registro de dispositivos

- Grupos de cosas
  - Las cosas se pueden agrupar hasta en 10 grupos de cosas
    - Son un método para facilitar la gestión de grandes cantidades de cosas a la vez, p.ej. políticas de seguridad, actualizaciones, cambios de localización, etc




  - Los grupos pueden organizarse en **jerarquías**
    - Las configuraciones hechas a nivel de grupo son heredadas por los objetos del grupo y de sus grupos descendientes





# Registro de dispositivos



- Uso del registro
  - Su uso es opcional
    - No es necesario dar de alta un dispositivo en el registro para conectarlo a AWS IoT pero sí para usar otras características como sombras, eventos o trabajos
  - La información sobre las cosas y tipos de cosas se almacenan en el registro de dispositivos de IoT Core como datos JSON

```json
{
    "version": 3,
    "thingName": "truckSensor01",
    "defaultClientId": "truckSensor01",
    "thingTypeName": "sensor_DoorAndPower",
    "attributes": {
        "deviceId": "T001",
        "powerRequirements": "12v"
    }
}
```

**Ejemplo 1: Cosa JSON**

```json
{
    "thingTypes": [
        {
            "thingTypeName": "sensor_DoorAndTemp",
            "thingTypeProperties": {
                "searchableAttributes": [
                    "deviceId",
                    "powerRequirements"
                ],
                "thingTypeDescription": "sensor for freezer trucks"
            },
            "thingTypeMetadata": {
                "deprecated": false,
                "creationDate": 1468423800950
            }
        }
    ]
}
```

**Ejemplo 2: Tipo de Cosa JSON**

  - Beneficios del uso del registro
    - Mejora la capacidad de búsqueda al mantener el listado de cosas, tipos y sus atributos
    - Permite enviar notificaciones cuando se crean, actualizan o eliminan cosas, tipos y grupos de cosas



# Contenidos

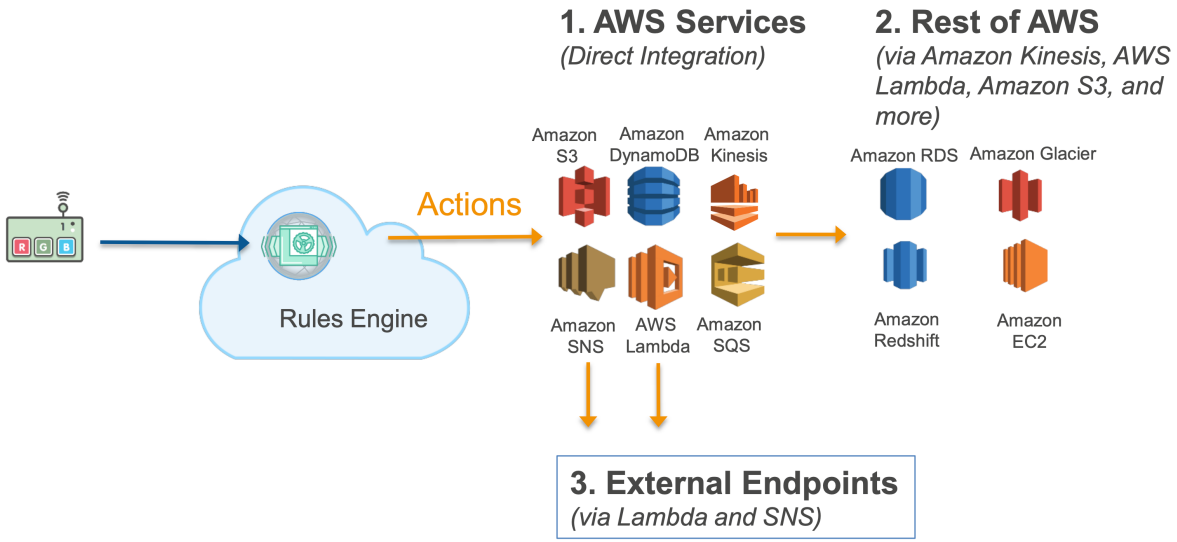
- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
    - Introducción
    - Seguridad
    - Registro de dispositivos
    - **Motor de reglas**
    - Sombras de dispositivo
  - Device Management
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



# Motor de reglas



- ¿Cómo funciona?
  - El motor de reglas filtra, transforma y encamina los datos con origen en el agente de mensajes a otros servicios AWS y de terceros
  - El motor aplica las reglas que seleccionan los datos de los mensajes MQTT y los transforman usando expresiones de tipo SQL y, finalmente, los encaminan usando alguna de las acciones predefinidas, p.ej. insertar los datos en una tabla DynamoDB, invocar una función Lambda, etc





# Motor de reglas



- Reglas
  - Una regla de AWS IoT está formada por
    - Nombre de la regla
    - Descripción (opcional)
    - Expresión SQL
      - Expresión para filtrar los mensajes entrantes en un tema MQTT y enviar los datos para otro lugar
      - Es buena práctica indicar la versión SQL que se está usando en la expresión
    - Una o más acciones
      - Acciones que van a ser ejecutadas al aplicar la regla
    - Una acción de error (opcional)
      - Acción ejecutada cuando no se consigue realizar la acción de la regla



# Motor de reglas



- Reglas
  - Una regla de AWS IoT está formada por
    - Nombre de la regla
    - Descripción (opcional)
    - Expresión SQL
    - Una o más acciones
    - Una acción de error (opcional)
  - **Ejemplo**

```
{
        "awsIotSqlVersion": "2016-03-23",
        "sql": "SELECT * FROM 'iot/test'",
        "ruleDisabled": false,
        "actions": [
                {
                        "s3": {
                                "roleArn": "arn:aws:iam::123456789012:role/aws_iot_s3",
                                "bucketName": "my-bucket",
                                "key": "myS3Key"
                        }
                }
        ]
}
```

**myrule.json**: declaración de una regla que inserta todos los mensajes enviados al tema `iot/test` en el *bucket* S3 `my-bucket`

```
aws iot create-topic-rule --rule-name myrule --topic-rule-payload file://myrule.json
```

Creación de la regla usando la CLI



# Motor de reglas



- Reglas
  - **Expresión SQL**
    - Usa una sintaxis SQL simplificada

**SELECT DATOS FROM TEMA WHERE FILTRO**

    - SELECT
      - Extrae información de la carga útil (*payload*) del mensaje entrante y la transforma
      - Los mensajes a procesar se escogen mediante el filtro de tema especificado en la cláusula FROM
    - FROM
      - El filtro de tema del mensaje MQTT que identifica los mensajes de los que extraer datos
      - La regla se activa para cada mensaje enviado a un tema MQTT que coincida con el filtro
      - Esta cláusula es opcional para reglas que solo se activen usando la ingestión básica
    - WHERE (opcional)
      - Condición lógica que determina si se ejecutan las acciones de la regla



# Motor de reglas



- Reglas
  - **Expresión SQL**
    - Usa una sintaxis SQL simplificada

**SELECT** **DATOS** **FROM** **TEMA** **WHERE** **FILTRO**

  - **Ejemplo**

```
{
 "color":"red",
 "temperature":100
}
```

```
SELECT color as rgb
FROM 'location01/sensor01'
WHERE temperature > 50
```

```
{
 "rgb":"red"
}
```

Tema MQTT: location01/sensor01

    - Más ejemplos: https://docs.aws.amazon.com/iot/latest/developerguide/iot-create-rule.html

# Motor de reglas

- Reglas
  - **Expresión SQL**
    - Usa una sintaxis SQL simplificada

**SELECT DATOS FROM TEMA WHERE FILTRO**

  - **Extensiones para JSON**
    - Operadores . y *
    - Extensiones a la sintaxis SQL estándar ANSI para facilitar el trabajo con objetos JSON anidados

- SELECT *
- SELECT deviceid, temp
- SELECT coords.latitude
- SELECT a.another_level.b
  - Returns {“b” : 3}
- SELECT a..b
  - Returns {“b” : 3}

```
{
      “deviceid” : “iot123”,
      “temp” : 54,
      “humidity” : 32,
      “coords” : {
            “latitude” : 47.615694,
            “longitude” : -122.3359976
      },
      “a” : {
            “another_level” : {
                  {“b” : 3},
                  {“b” : 5}
      }}
}
```

```
{
  "data": {
    "item2": {
      "0": {
        "key": "Value"
      }
    }
  }
}
```

Tema MQTT: topic/subtopic

```
SELECT *
FROM 'topic/subtopic'
WHERE data.item2.0.key= "Value"
```



# Motor de reglas



- Reglas
  - **Expresión SQL**
    - Usa una sintaxis SQL simplificada

**SELECT** **DATOS** **FROM** **TEMA** **WHERE** **FILTRO**

  - **Funciones**
    - Disponibles muchos tipos de funciones, p.ej. matemáticas, lógicas, manipulación de *strings*, consulta y transformación de mensajes, invocación de funciones Lambda, codificación/decodificación de mensajes, acceso a tablas DynamoDB, predicciones con modelos ML de SageMaker

```
SELECT *, clientId() as MQTTClientId
FROM 'one/rule'
WHERE
startsWith(topic(2), 'Vac123') AND
(state = 'SWEEP' OR bin.size < 30)"
```

```
SELECT aws_lambda("arn:aws:lambda:us-east-1:account_id:function:lambda_function", {"payload":attribute1}) as output
FROM 'topic'
```

Invocación de una función Lambda (tiempo máximo: 2s)

```
{
 "attribute1": 21,
 "attribute2": "value"
}
```

Tema MQTT: topic



# Motor de reglas



- Reglas
  - **Expresión SQL**
    - Usa una sintaxis SQL simplificada

**SELECT DATOS FROM TEMA WHERE FILTRO**

  - **Plantillas de sustitución**
    - Sirven para modificar los datos JSON devueltos por una regla que es activada y ejecuta una acción
    - **Sintaxis**: ${ expresión}
      - La expresión puede ser una de las soportadas en las cláusulas SELECT y WHERE y en las acciones de la regla
    - Las plantillas permiten configurar dinámicamente los campos de las acciones de las reglas
      - Los campos que pueden modificarse varían con cada tipo de acción

```
{
    "deviceid": "iot123",
    "temp": 54.98,
    "humidity": 32.43,
    "coords": {
        "latitude": 47.615694,
        "longitude": -122.3359976
    }
}
```

Tema MQTT: my/iot/topic

```
{
    "sql": "SELECT *, timestamp() AS timestamp FROM 'my/iot/topic'",
    "ruleDisabled": false,
    "actions": [{
        "republish": {
            "topic": "${topic()}/republish",
            "roleArn": "arn:aws:iam::123456789012:role/my-iot-role"
        }
    }]
}
```

Regla con acción de tipo volver a publicar en un tema MQTT (*republish*)

```
{
    "deviceid": "iot123",
    "temp": 54.98,
    "humidity": 32.43,
    "coords": {
        "latitude": 47.615694,
        "longitude": -122.3359976
    },
    "timestamp": 1579637878451
}
```

Tema MQTT: my/iot/topic/republish



# Motor de reglas



- Reglas
  - **Expresión SQL**
    - Usa una sintaxis SQL simplificada

**SELECT** **DATOS** **FROM** **TEMA** **WHERE** **FILTRO**

  - **Plantillas de sustitución**
    - Sirven para modificar los datos JSON devueltos por una regla que es activada y ejecuta una acción
    - **Sintaxis**: ${ expresión}
      - La expresión puede ser una de las soportadas en las cláusulas SELECT y WHERE y en las acciones de la regla
    - Las plantillas permiten configurar dinámicamente los campos de las acciones de las reglas
      - Los campos que pueden modificarse varían con cada tipo de acción

```
{
    "topicRulePayload": {
        "sql": "SELECT * FROM 'some/topic'",
        "ruleDisabled": false,
        "awsIotSqlVersion": "2016-03-23",
        "actions": [
            {
                "s3": {
                    "bucketName": "my-bucket",
                    "cannedacl": "public-read",
                    "key": "${topic()}/${timestamp()}",
                    "roleArn": "arn:aws:iam::123456789012:role/aws_iot_s3"
                }
            }
        ]
    }
}
```

Regla que guarda los datos en S3, en el *bucket* **my-bucket** con la clave de objeto creada dinámicamente con el nombre del tema MQTT y el *timestamp*



# Motor de reglas



- Reglas
  - **Acciones**
    - Las acciones especifican que hacer cuándo se acciona una regla
    - AWS IoT soporta las siguientes acciones

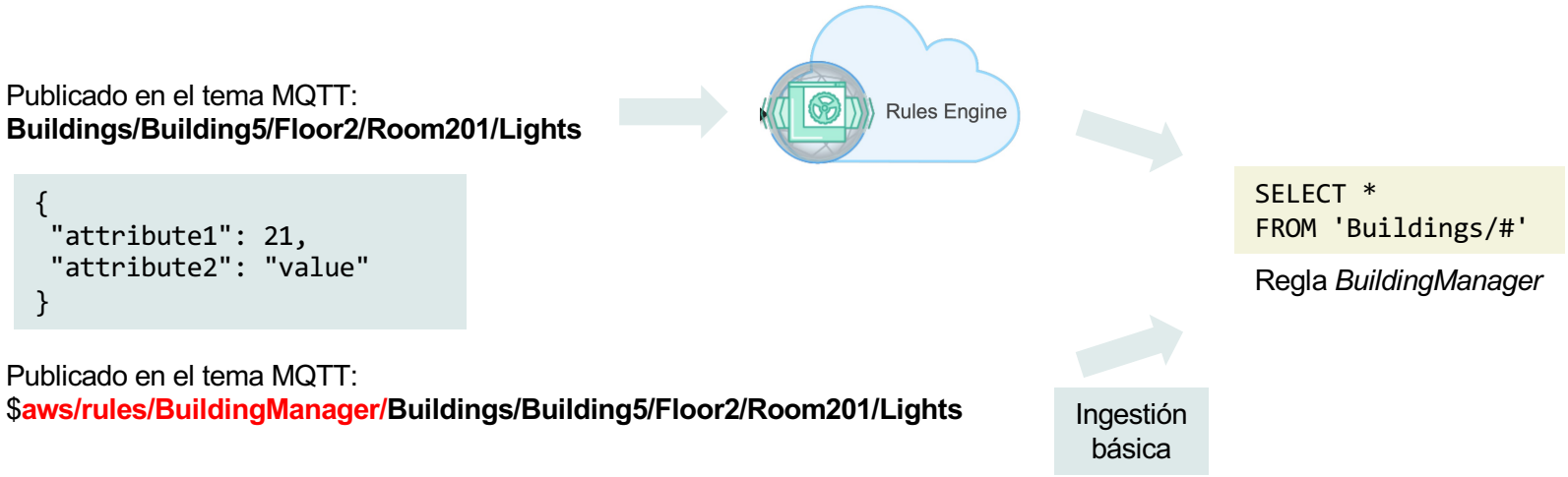
| Acción de regla | Descripción | Nombre en la API |
|---|---|---|
| Apache Kafka | Envía un mensaje a un clúster de Apache Kafka. | kafka |
| CloudWatch alarmas | Cambia el estado de una CloudWatch alarma de Amazon. | cloudwatchAlarm |
| CloudWatch Registros | Envía un mensaje a Amazon CloudWatch Logs. | cloudwatchLogs |
| CloudWatch métricas | Envía un mensaje a una CloudWatch métrica. | cloudwatchMetric |
| DynamoDB | Envía un mensaje a una tabla DynamoDB. | dynamoDB |
| DynamoDBv2 | Envía los datos de los mensajes a varias columnas de una tabla de DynamoDB. | dynamoDBv2 |
| Elasticsearch | Envía un mensaje a un OpenSearch punto final. | OpenSearch |
| HTTP | Publica un mensaje en un punto de conexión HTTPS. | http |
| IoT Analytics | Envía un mensaje a un AWS IoT Analytics canal. | iotAnalytics |
| AWS IoT Events | Envía un mensaje a una AWS IoT Events entrada. | iotEvents |
| AWS IoT SiteWise | Envía los datos del mensaje a las propiedades AWS IoT SiteWise de los activos. | iotSiteWise |
| Firehose | Envía un mensaje a una cadena de entrega de Firehose. | firehose |
| Kinesis Data Streams | Envía un mensaje a un flujo de datos Kinesis. | kinesis |
| Lambda | Invoca una función de Lambda con datos de mensaje como entrada. | lambda |
| Ubicación | Envía los datos de ubicación a Amazon Location Service. | location |
| OpenSearch | Envía un mensaje a un punto final OpenSearch de Amazon Service. | OpenSearch |
| Republish | Vuelve a publicar un mensaje en otro tema MQTT. | republish |
| S3 | Almacena un mensaje en un bucket de Amazon Simple Storage Service (Amazon S3). | s3 |
| Salesforce IoT | Envía un mensaje a un flujo de entrada de Salesforce IoT. | salesforce |
| SNS | Publica un mensaje como notificación push de Amazon Simple Notification Service (Amazon SNS). | sns |
| SQS | Envía un mensaje a una cola de Amazon Simple Queue Service (Amazon SQS). | sqs |
| Step Functions | Inicia una máquina de AWS Step Functions estados. | stepFunctions |
| Timestream | Envía un mensaje a una tabla de base de datos de Amazon Timestream. | timestream |



# Motor de reglas



- Ingestión básica
  - Los mensajes se ingieren directamente al motor de reglas sin pasar por el agente de mensajes
  - Permite reducir los costes de ingestión de mensajes manteniendo el nivel de seguridad
  - Los mensajes se publican en temas reservados cuyos nombres comienzan por `$aws/rules/rule_name`, donde `rule_name` es el nombre de la regla que se quiere invocar





# Contenidos

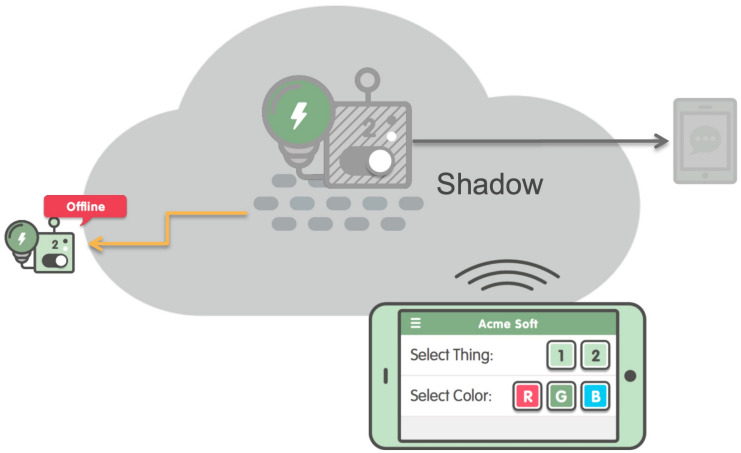
- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
    - Introducción
    - Seguridad
    - Registro de dispositivos
    - Motor de reglas
    - **Sombras de dispositivo**
  - Device Management
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



# Sombras de dispositivo

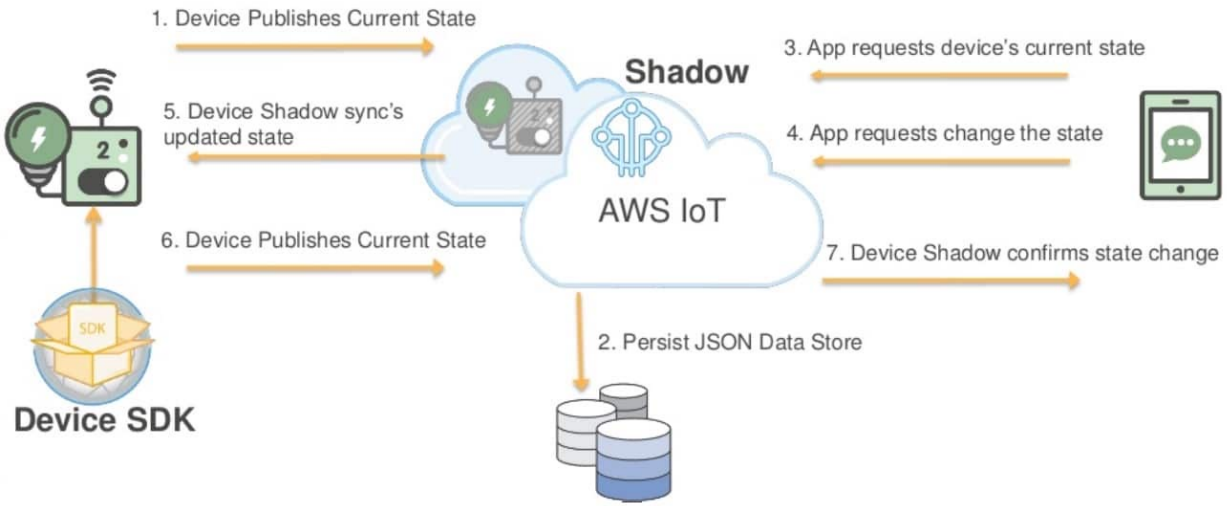


- ¿Qué es una sombra?
  - Una sombra es una representación en AWS IoT del estado de un dispositivo
  - Hacen disponible el estado del dispositivo para aplicaciones y otros servicios, que pueden acceder y controlar el estado actual del dispositivo a través de sus sombras tanto si el dispositivo está conectado como si no



# Sombras de dispositivo

- Usando sombras
  - **Ejemplo de funcionamiento**
    1. Un dispositivo comunica su estado actual
    2. El servicio de sombras actualiza las sombras con el estado actual del dispositivo
    3. Una aplicación puede suscribirse a la actualización de las sombras o consultar su estado
    4. La aplicación solicita un cambio en el estado del dispositivo actualizando sus sombras
    5. El servicio de sombras notifica un cambio en el estado de las sombras
    6. El dispositivo recibe esta notificación, sincroniza su estado con el de las sombras y comunica su estado actualizado
    7. El servicio de sombras refleja el estado actualizado en las sombras





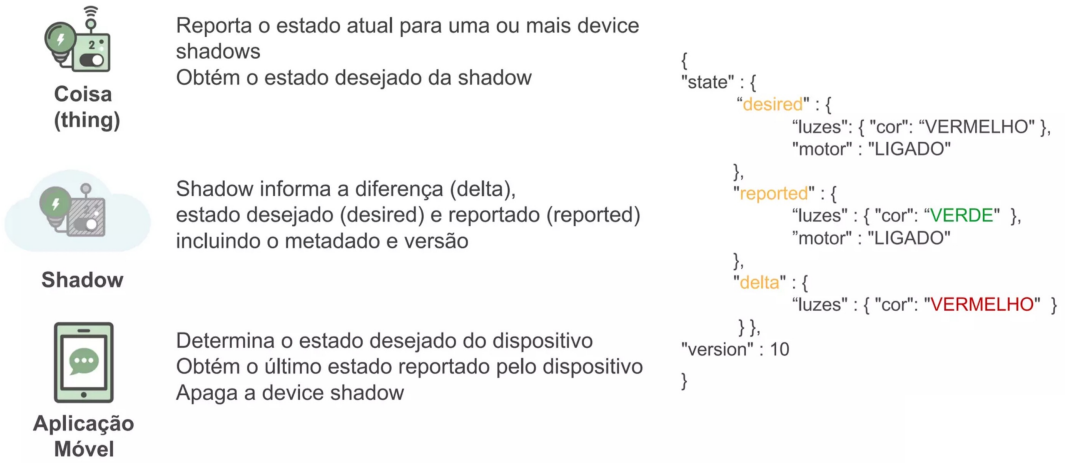
# Sombras de dispositivo

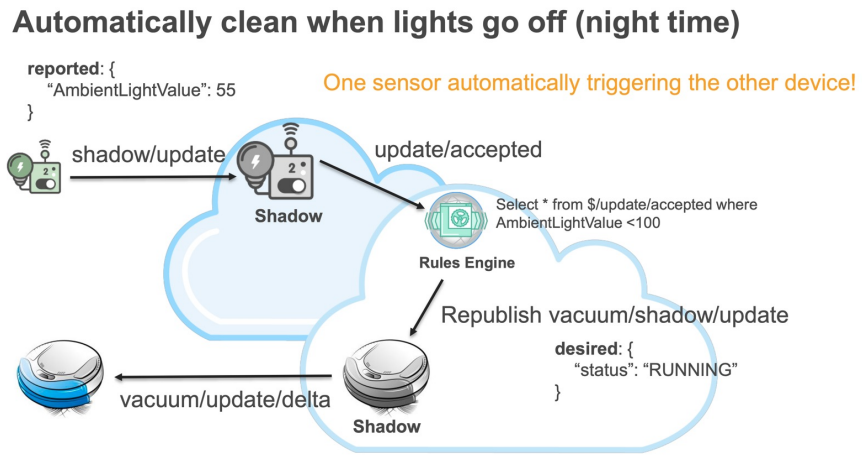


- Usando sombras
  - **Ejemplo de funcionamiento**
    1. Un dispositivo comunica su estado actual
    2. El servicio de sombras actualiza las sombras con el estado actual del dispositivo
    3. Una aplicación puede suscribirse a la actualización de las sombras o consultar su estado
    4. La aplicación solicita un cambio en el estado del dispositivo actualizando sus sombras
    5. El servicio de sombras notifica un cambio en el estado de las sombras
    6. El dispositivo recibe esta notificación, sincroniza su estado con el de las sombras y comunica su estado actualizado
    7. El servicio de sombras refleja el estado actualizado en las sombras
  - **Si el dispositivo se queda sin conexión**, la aplicación aún puede comunicarse con sus sombras
    - Cuando el dispositivo se reconecta, recibe el estado actual de sus sombras para que pueda sincronizar su estado con el de las sombras
  - Del mismo modo, **si la aplicación deja de estar en línea** y el estado del dispositivo cambia mientras tanto, el dispositivo mantiene las sombras actualizadas para que la aplicación pueda consultar su estado actual cuando se reconecte

# Sombras de dispositivo

- Usando sombras
  - El uso de sombras **requiere coordinación** entre dispositivos, aplicaciones y otros servicios en la nube con permiso para acceder a las sombras para mantener la consistencia entre el estado de las sombras y el del dispositivo
    - El servicio de sombras almacena el estado de las sombras, envía mensajes cuando el estado de las sombras cambia y responde a mensajes que cambian su estado
    - Los dispositivos deben comunicar datos de su estado a las sombras modificando solo la propiedad `reported`
    - Las aplicaciones y otros servicios en la nube deben comunicar solicitudes de cambio de estado al dispositivo a través de las sombras modificando solo la propiedad `desired`





# Sombras de dispositivo



- Acceso a las sombras
  - La comunicación de los dispositivos con las sombras se hace usando MQTT
  - Cada sombra tiene
    - Un tema MQTT general reservado
      - $aws/things/thingName/shadow
    - Una URL HTTP que soporta las acciones GET, UPDATE y DELETE en la sombra
      - https://account-specific-prefix-ats.iot.region.amazonaws.com/things/thingName/shadow
    - Cada acción (GET, UPDATE, DELETE) tiene un tema de solicitud, uno de respuesta correcta (accepted) y otro de respuesta incorrecta (rejected)
  - Se necesita autorización para poder publicar y suscribirse en los temas de la sombra

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iot:Subscribe"
      ],
      "Resource": [
        "arn:aws:iot:region:account:topicfilter/$aws/things/thingName/shadow/get/accepted"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "iot:Receive"
      ],
      "Resource": [
        "arn:aws:iot:region:account:topic/$aws/things/thingName/shadow/get/accepted"
      ]
    }
  ]
}
```

**Tópicos**

- /get
- /get/accepted
- /get/rejected
- /update
- /update/delta
- /update/accepted
- /update/documents
- /update/rejected
- /delete
- /delete/accepted
- /delete/rejected



# Sombras de dispositivo



- Tipos de sombras
  - Cada cosa puede tener
    - Una única sombra clásica (sin nombre)
      - Son más simples pero algo más limitadas que las sombras con nombre
    - Múltiples sombras con nombre
      - Permiten crear diferentes vistas del estado de un dispositivo
      - Ejemplos de uso
        - Dividir una cosa con muchas propiedades en sombras con las propiedades organizadas en diferentes grupos, cada uno identificado por su nombre de sombra
        - Limitar el acceso a las propiedades de la cosa agrupándolas en diferentes sombras y empleando políticas para controlar el acceso
  - Los temas MQTT utilizados por cada tipo difieren solo en el prefijo del tema

| *ShadowTopicPrefix*valor | Tipo de sombra |
|---|---|
| $aws/things/*thingName*/shadow | Sombra sin nombre (clásica) |
| $aws/things/*thingName*/shadow/name/*shadowName* | Sombra con nombre |



# Sombras de dispositivo



- Documentos de sombra
  - Las sombras usan documentos JSON para almacenar y recuperar datos
  - **Tipos** (cada uno tiene su formato propio)
    - Solicitud de estado
    - Respuesta de estado
      - /accepted, /delta y /documents
    - Respuesta de error
    - Respuesta de lista de nombres de sombra
  - Control de versiones
    - El servicio de sombras soporta el uso de versiones en los mensajes de actualización
    - En cada actualización de una sombra la versión del documento JSON es incrementada
    - Las versiones aseguran dos cosas
      - Un cliente recibe un error si intenta modificar una sombra usando un número de versión antiguo. El cliente debe sincronizarse con la última versión antes de poder actualizar la sombra
      - Un cliente puede ignorar los mensajes recibidos que tengan versiones inferiores a la de la copia del documento que él almacena
    - El control de versiones se ignora si no se incluye la propiedad `version` en el documento de sombra



# Sombras de dispositivo



- Documentos de sombra
  - Las sombras usan documentos JSON para almacenar y recuperar datos
  - **Propiedades de un documento de sombra**

```
{
    "state": {
        "desired": {
            "attribute1": integer2,
            "attribute2": "string2",
            ...
            "attributeN": boolean2
        },
        "reported": {
            "attribute1": integer1,
            "attribute2": "string1",
            ...
            "attributeN": boolean1
        }
    },
    "clientToken": "token",
    "version": version
}
```

- state
  - desired
    - El estado deseado del dispositivo
    - Las aplicaciones modifican esta propiedad para actualizar el estado del dispositivo sin tener que conectarse a él
  - reported
    - El estado actual comunicado por el dispositivo
  - delta
    - Diferencia entre el estado desired y el reported
- metadata
  - Sigue el mismo esquema que state
  - Contiene los valores UTC que indican cuando se actualizaron los valores de state por última vez
- version
  - Valor entero que se incrementa cada vez que se hace una modificación del documento
  - Permite que las aplicaciones y los dispositivos sepan si su copia local del documento es la última
- timestamp
  - Indica el horario UTC del envío de la actualización del documento desde AWS IoT
- clientToken
  - Un *string* exclusivo de cada dispositivo que permite asociar respuestas con solicitudes



# Contenidos

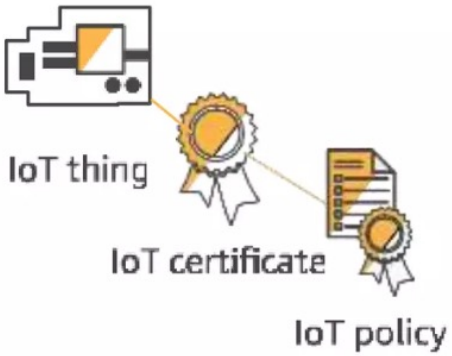
- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - **Device Management**
    - **Aprovisionamiento de dispositivos**
    - Trabajos remotos
    - Indexación de flotas
    - Tunelización segura
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos

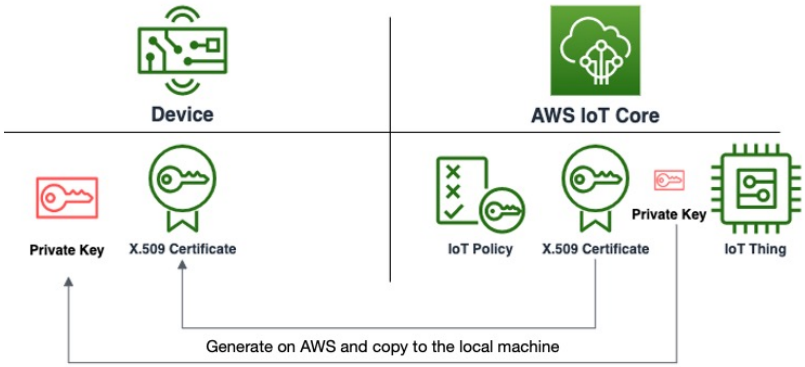


# Aprovisionamiento de dispositivos



- Proceso de aprovisionamiento de dispositivos
  - Para aprovisionar un dispositivo y que pueda comunicarse con seguridad con AWS IoT hay que hacer lo siguiente
    - Registrar la **cosa** que va a representar al dispositivo
    - Obtener un **certificado X.509** y asociarlo al dispositivo
      - Puede registrarse un certificado existente o dejar que sea AWS IoT quien lo genere
      - El certificado se asocia con la cosa y tiene que copiarse en el dispositivo junto con la clave privada
    - Configurar una **política IoT** y asociarla al certificado
      - Define las operaciones que el dispositivo puede realizar en AWS IoT cuando se identifica con el certificado




Proceso de aprovisionamiento de dispositivos durante el desarrollo usando IoT Core. No debe usarse para producción ya que no es seguro.



# Aprovisionamiento de dispositivos



- Plantillas de aprovisionamiento
  - Documentos JSON usados para el aprovisionamiento automático de dispositivos
    - Describen los recursos que IoT Core tiene que crear para aprovisionar los dispositivos
    - Pueden parametrizarse para aprovisionar múltiples dispositivos con la misma plantilla
    - Tienen dos secciones
      - **Recursos:** define la cosa, el certificado y una o más políticas IoT para aprovisionar el dispositivo
      - **Parámetros**: define los parámetros usados en la sección Recursos. Sintaxis: `{"Ref":"nombre-parámetro"}`

```
{
    "Resources" : {
        "thing" : {
            "Type" : "AWS::IoT::Thing",
            "Properties" : {
                "ThingName" : {"Ref" : "ThingName"},
                "AttributePayload" : { "version" : "v1", "serialNumber" :  {"Ref" : "SerialNumber"}},
                "ThingTypeName" :  "lightBulb-versionA",
                "ThingGroups" : ["v1-lightbulbs", {"Ref" : "Location"}]
            },
            "OverrideSettings" : {
                "AttributePayload" : "MERGE",
                "ThingTypeName" : "REPLACE",
                "ThingGroups" : "DO_NOTHING"
            }
        },
        "certificate" : {
            "Type" : "AWS::IoT::Certificate",
            "Properties" : {
                "CertificateSigningRequest": {"Ref" : "CSR"},
                "Status" : "ACTIVE"
            }
        },
        "policy" : {
            "Type" : "AWS::IoT::Policy",
            "Properties" : {
                "PolicyDocument" : "{ \"Version\": \"2012-10-17\", \"Statement\": [{ \"Effect\": \"Allow\", \"Action\":[\"iot:Publish\"], \"Resource\": [\"arn:aws:iot:us-east-1:123456789012:topic/foo/bar\"] }] }"
            }
        }
    }
}
```



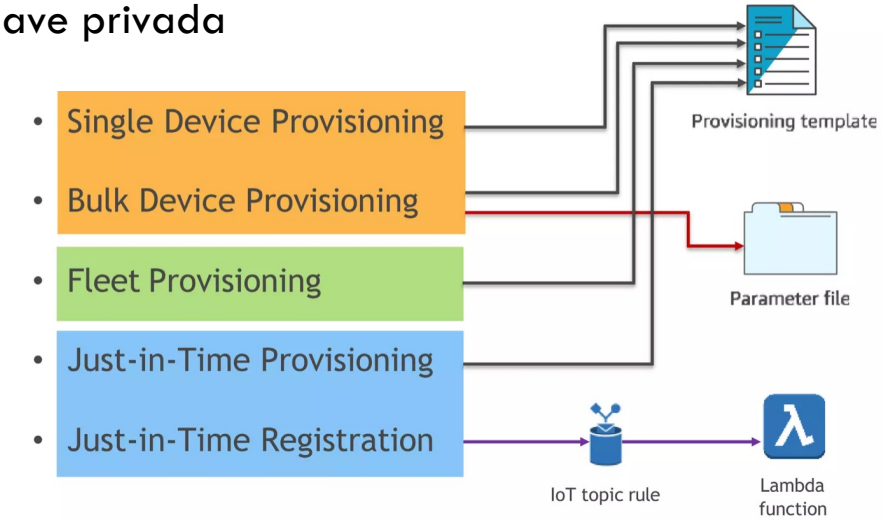
# Aprovisionamiento de dispositivos



- Métodos de aprovisionamiento
  - El aprovisionamiento de un dispositivo puede ser manual o automático
  - Opciones en AWS IoT para el **aprovisionamiento autómatico de dispositivos**
    - Si el dispositivo ya viene con certificado y clave privada de fábrica
      - Aprovisionamiento individual
        - Los parámetros de la plantilla de aprovisionamiento se especifican en el momento del registro de la cosa en IoT
      - Registro en masa (*Bulk provisioning*)
        - Se proporciona en S3 la lista con los parámetros de la plantilla de aprovisionamiento de los dispositivos

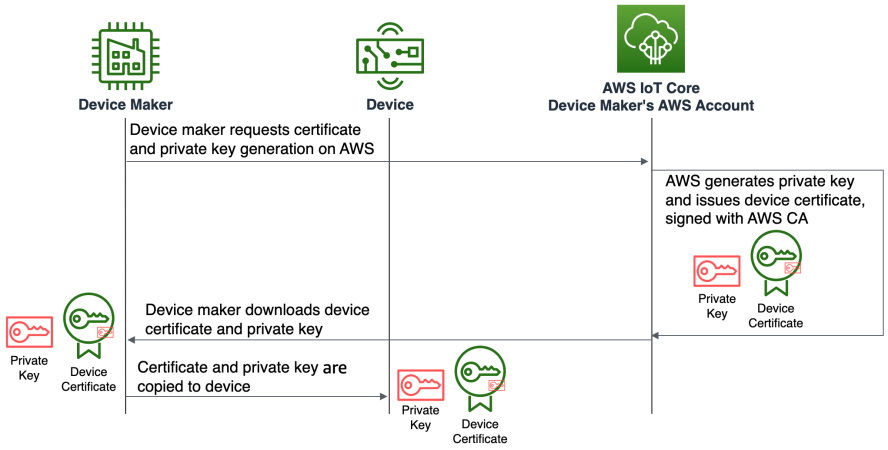
```
{"ThingName": "foo", "SerialNumber": "123", "CSR": "csr1"}
{"ThingName": "bar", "SerialNumber": "456", "CSR": "csr2"}
```

      - *Just-In-Time Provisioning* (JITP)/*Just-In-Time Registration* (JITR)
    - Si el dispositivo no dispone de certificado y clave privada
      - Aprovisionamiento de flota (*Fleet provisioning*)

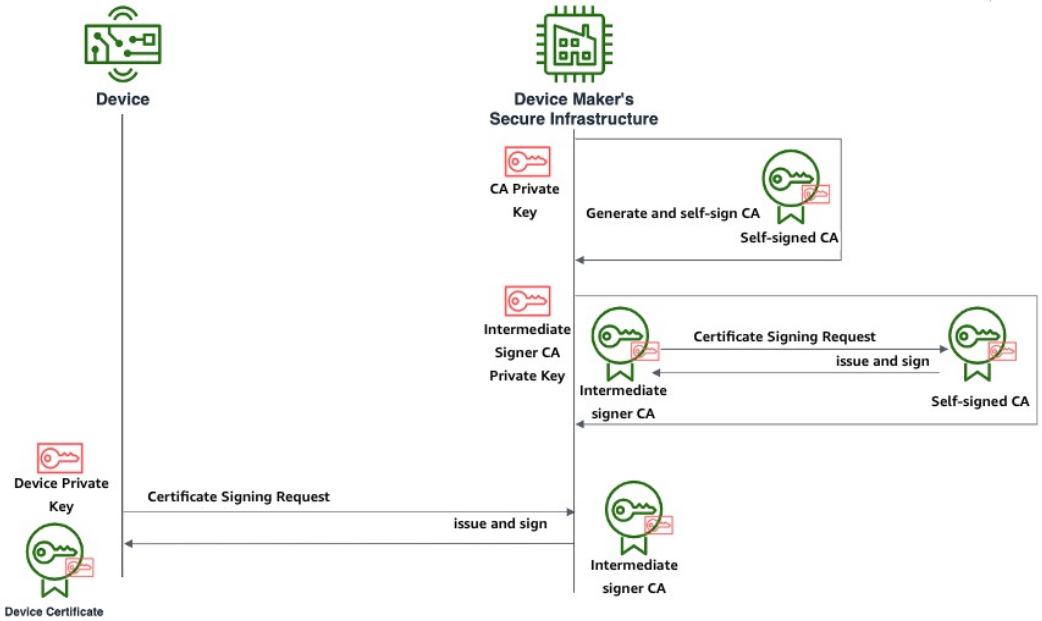


# Aprovisionamiento de dispositivos

- Gestión de la infraestructura de clave pública (PKI)
  - La infraestructura de clave pública (PKI, *Public Key Infrastructure*) está formada por las Autoridades de Certificación (CA) que emiten y firman los certificados X.509
  - En AWS IoT los fabricantes de dispositivos pueden escoger entre usar AWS IoT como CA, usar una CA propia o usar una CA de terceros
    - **Ejemplos**




Certificado X.509 y clave privada generada por AWS



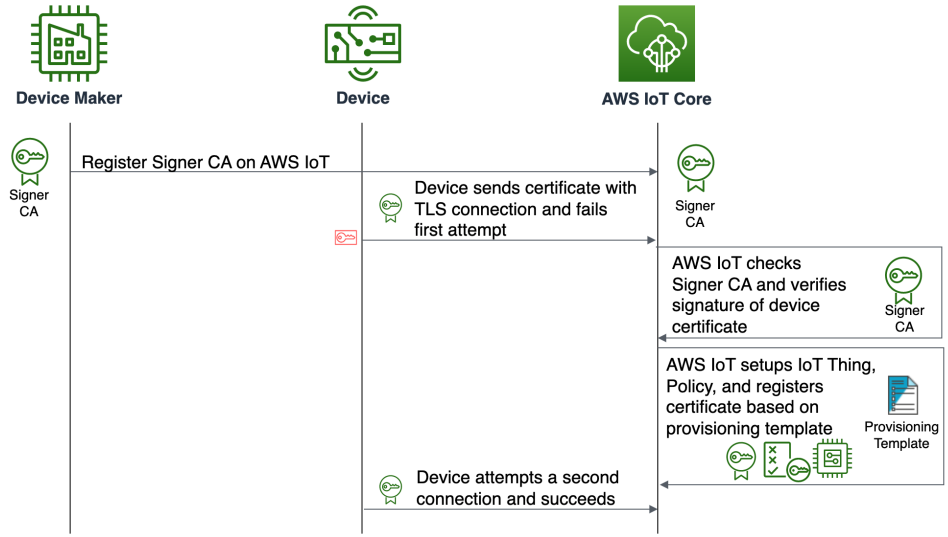

Infraestructura PKI propia del fabricante de dispositivos



# Aprovisionamiento de dispositivos



- Just-In-Time Provisioning (JITP)
  - Los dispositivos ya disponen de certificado y clave privada
  - El fabricante de los dispositivos tiene que registrar la CA con la que se firmaron los certificados en una cuenta AWS y proporcionar una plantilla de aprovisionamiento
    - No se pueden usar los certificados generados por AWS IoT para JITP
    - El fabricante debe saber a qué cuenta AWS se van a conectar los dispositivos
  - En la primera conexión del dispositivo a IoT Core
    - Se verifica la firma del certificado del dispositivo contra la CA registrada
    - Se usa la plantilla de aprovisionamiento del fabricante para registrar la cosa y el certificado y asignarle una política IoT al dispositivo

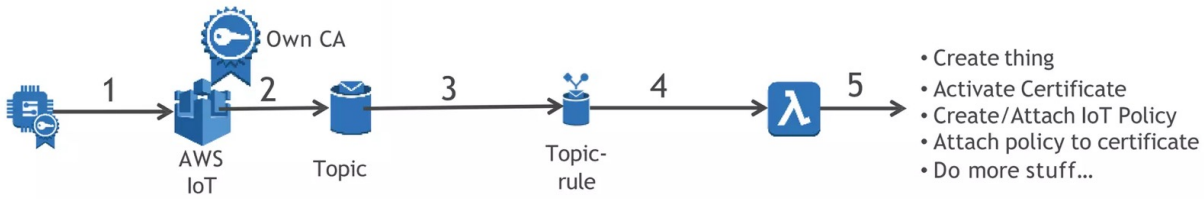




# Aprovisionamiento de dispositivos



- Just-In-Time Registration (JITR)
  - Usa una función Lambda para añadir lógica personalizada al método JITP
  - La función es invocada por una regla IoT configurada por el fabricante para los eventos notificados por IoT Core en un tema MQTT reservado cuando se verifica la firma del certificado de un dispositivo




`$aws/events/certificates/registered/<caCertificateID>`

  - La función Lambda es la que crea la cosa, configura las políticas IoT y las asocia con el certificado. Además puede realizar otras acciones como
    - Una validación secundaria contra una lista de permisos o lista de revocación de certificados (CRL)
    - Registrar el dispositivo para un usuario en particular
    - Realizar otras operaciones de aprovisionamiento adicionales



# Aprovisionamiento de dispositivos



- Métodos de aprovisionamiento

## JITR vs. JITP

| JITR | JITP |
|---|---|
| Topic rule and Lambda function. Code must be written and maintained | No code, only body template attached to CA |
| Provisioning more complex: Device connects, certificate registers with status PENDING_ACTIVATION, service sends MQTT message, rule triggers Lambda, Lambda does provisioning and optionally more stuff | Easy provisioning: Device connects, provisioning workflow run automatically |
| Flexible, different policies for different devices can be created/attached. Information from/to the provisioning process can be put/read from other systems, etc. | Static, same provisioning process for every device |



# Aprovisionamiento de dispositivos



- Métodos de aprovisionamiento
  - **Aprovisionamiento de dispositivos que no dispongan de certificados**
    - No siempre es posible aprovisionar credenciales únicas a un dispositivo durante la fabricación
      - Causas: limitaciones técnicas, costes o requisitos específicos de la aplicación
      - Estos dispositivos llegan a los usuarios sin disponer de una identidad única
    - El **aprovisionamiento de flota** (*fleet provisioning*) genera una clave privada y el certificado de dispositivo firmado por la CA de AWS cuando el dispositivo se conecta por primera vez
    - Hay dos métodos
      - **Aprovisionamiento por usuario de confianza** (*provisioning by trusted user*)
        - Los usuarios o instaladores disponen de una aplicación móvil proporcionada por el fabricante para obtener unas credenciales temporales para el aprovisionamiento del dispositivo
      - **Aprovisionamiento por reclamación** (*provisioning by claim*)
        - El fabricante instala un mismo certificado con permisos limitados (**certificado de reclamación**) y clave privada en todos los dispositivos de una flota
        - Los dispositivos usan ese certificado en la primera conexión para aprovisionar una identidad única
        - Esta opción presenta un riesgo mayor que las otras si el certificado de reclamación se ve comprometido



# Contenidos

- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - **Device Management**
    - Aprovisionamiento de dispositivos
    - **Trabajos remotos**
    - Indexación de flotas
    - Tunelización segura
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



# Trabajos remotos



- ¿Qué es un trabajo remoto?
  - Una **operación remota** es cualquier actualización o acción que se puede hacer de forma remota en un dispositivo físico, virtual o *endpoint* sin necesidad de la presencia física de un operador o técnico, p.ej.
    - Descarga e instalación de aplicaciones
    - Actualizaciones OTA de *firmware*
    - Reinicio de dispositivos
    - Restablecimiento de la configuración de fábrica
    - Rotación de certificados
  - Los **trabajos** son conjuntos de operaciones remotas





# Trabajos remotos



- Conceptos básicos
  - **Documento del trabajo**
    - Documento JSON con la información que los dispositivos necesitan para ejecutar el trabajo
    - Contiene la(s) URL desde donde el dispositivo puede descargar actualizaciones u otros datos
    - El documento puede estar almacenado en un *bucket* S3 o incluirse como argumento del comando que crea el trabajo
  - **Despliegue**
    - Despliegue de un documento de trabajo en una lista de dispositivos destino
  - **Ejecución de un trabajo**
    - Instancia de la ejecución de un trabajo en un dispositivo destino
    - Tiene asociado un identificador único
  - **Tipos de trabajos**
    - Trabajo instantáneo (*snapshot job*)
      - Por defecto se envía el trabajo a todos los destinos que se especifiquen cuando se crea
      - Termina una vez que los destinos completen el trabajo o informen de que no pueden hacerlo
    - Trabajo continuo (*continuous job*)
      - Se envía el trabajo a todos los destinos que se especifiquen cuando se crea
      - Sigue activo y se envía a los nuevos dispositivos que se añadan al grupo de destino



# Trabajos remotos



- Plantillas de trabajos
  - Permiten preconfigurar trabajos con valores estándar para aquellas acciones remotas que se realicen con frecuencia en diferentes conjuntos de dispositivos de destino
  - Una plantilla define las siguientes configuraciones y recursos
    - Propiedades del trabajo
    - Documento del trabajo y destinos
    - Configuraciones de despliegue, programación (*scheduling*) y cancelación
    - Configuraciones de tiempo de espera y reintento
  - Una plantilla puede ser de dos tipos
    - Personalizada
      - El usuario es quien define la plantilla
    - Gestionada por AWS IoT
      - Estas plantillas proporcionan un documento de trabajo predefinido para acciones remotas comunes
      - Facilitan la creación de trabajos reutilizables



# Trabajos remotos



- Plantillas gestionadas por AWS
  - Estas plantillas contienen
    - El entorno en el que ejecutar los comandos del documento del trabajo
    - Un documento del trabajo que especifica el nombre y parámetros de la operación remota
      - Los parámetros dependen de la operación concreta excepto `runAsUser` y `pathToHandler` que son comunes a todos
      - Operaciones: `Download-File`, `Install-Application`, `Remove-Application`, `Start-Application`, `Restart-Application`, `Stop-Application`, `Run-Command`, `Reboot`
  - Ejemplo: **`Download-File`**

```
{
  "version": "1.0",
  "steps": [
    {
      "action": {
        "name": "Download-File",
        "type": "runHandler",
        "input": {
          "handler": "download-file.sh",
          "args": [
            "${aws:iot:parameter:downloadUrl}",
            "${aws:iot:parameter:filePath}"
          ],
          "path": "${aws:iot:parameter:pathToHandler}"
        },
        "runAsUser": "${aws:iot:parameter:runAsUser}"
      }
    }
  ]
}
```

Nombre de la operación: *Download-File*
Parámetros:

- *runAsUser*
  Usuario en el dispositivo que se usará para ejecutar el trabajo
- *pathToHandler*
  La ruta local en el dispositivo del agente (*handler*) que ejecutará el trabajo
- *downloadUrl*
  La URL del fichero a descargar
- *filePath*
  La ruta local en el dispositivo en la que descargar el fichero



# Trabajos remotos



- Dispositivos y trabajos
  - Los dispositivos pueden comunicarse con el servicio de trabajos usando
    - MQTT, HTTP Signature Version 4 o HTTP TLS
  - **Uso del protocolo MQTT**
    - Los dispositivos se suscriben a temas MQTT para
      - Ser notificados de la presencia de nuevos trabajos
      - Recibir respuestas del servicio de trabajos
    - Los dispositivos publican en temas MQTT para
      - Consultar o actualizar el estado de un trabajo
    - Cada dispositivo tiene su propio tema MQTT general y puede suscribirse a los siguientes
      - $aws/things/thing-name/jobs/notify
        - Para notificaciones de cuándo se añade o elimina un trabajo a la lista de trabajos pendientes
      - $aws/things/thing-name/jobs/notify-next
        - Para notificaciones de cuándo cambió la primera ejecución de un trabajo en la lista de trabajos pendientes
      - $aws/things/thing-name/jobs/request-name/accepted
      - $aws/things/thing-name/jobs/request-name/rejected
      - $aws/things/thing-name/jobs/jobId/request-name/accepted
      - $aws/things/thing-name/jobs/jobId/request-name/rejected
        - Para mensajes de éxito y fracaso de una solicitud (p.ej. GET, UPDATE)



# Trabajos remotos



- Dispositivos y trabajos
  - **Uso del protocolo MQTT**
    - Ejemplo
      - Cuando se crea un nuevo trabajo, AWS IoT publica para cada dispositivo de destino un mensaje en el tema $aws/things/thing-name/jobs/notify
      - Para obtener más información sobre la ejecución del trabajo, el dispositivo llama a la función `DescribeJobExecution` de la API MQTT y, si la solicitud tiene éxito, AWS IoT publica un mensaje en el tema $aws/things/thing-name/jobs/jobId/get/accepted

```
{
    "clientToken" : "client-001",
    "timestamp" : 1489097434407,
    "execution" : {
        "approximateSecondsBeforeTimedOut": number,
        "jobId" : "023",
        "status" : "QUEUED",
        "queuedAt" : 1489097374841,
        "lastUpdatedAt" : 1489097374841,
        "versionNumber" : 1,
        "jobDocument" : {
            < contents of job document >
        }
    }
}
```

      - El dispositivo cuenta ahora con el documento del trabajo que usa para ejecutar las acciones remotas definidas en él



# Contenidos

- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - **Device Management**
    - Aprovisionamiento de dispositivos
    - Trabajos remotos
    - **Indexación de flotas**
    - Tunelización segura
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos



# Indexación de flotas



- Permite organizar, buscar y solucionar problemas en una flota de dispositivos
  - Indexa y agrega datos de los dispositivos desde distintas fuentes de AWS IoT, p.ej.
    - El registro de dispositivos, Device Shadow, el catálogo de paquetes software o las violaciones de Device Defender
  - Facilita consultar un grupo de dispositivos y agregar estadísticas que combinen sus atributos, incluyendo el estado, conectividad y violaciones de Defender
  - Características
    - Gestión de actualizaciones del índice
      - Se pueden configurar índices para indexar actualizaciones en los grupos de cosas, registros de cosas, sombras de dispositivos, conectividad, localización y violaciones de Defender
      - AWS IoT actualiza los índices continuamente con los últimos datos de los dispositivos
    - Búsquedas en múltiples fuentes de datos
      - Se pueden realizar consultas a través de múltiples fuentes de datos usando un lenguaje de consulta que describe las cosas que se quieren encontrar
    - Consulta de datos agregados
      - Las búsquedas pueden devolver estadísticas de los datos agregados: media, percentil, varianza, etc
    - Creación de alarmas usando métricas de flota
      - Se pueden definir métricas de flota para enviar datos agregados a CloudWatch automáticamente, analizar tendencias y crear alarmas para monitorizar el estado de la flota, p.ej. el ratio de desconexión de los dispositivos o el promedio de cambios de nivel de la batería en un período dado



# Indexación de flotas



- Ejemplos de consultas
  - Sintaxis basada en la de Lucene (no soporta todas las características)
    - thingName:Sensor01 // Cosa de nombre Sensor01
    - thingName:Sensor* // Cosas con nombres que comiencen por “Sensor”
    - thingName:Sensor0? // Cosas con nombres que comiencen por “Sensor0” y tengan un carácter más, p.ej. Sensor01, Sensor0A, etc
    - thingTypeName:Sensors // Cosas de tipo “Sensors”
    - attributes.temp:[25 TO 30] // Cosas con valor del atributo “temp” en el rango [25, 30]
    - attributes.temp:{25 TO 30] // Cosas con valor del atributo “temp” en el rango (25, 30]
    - shadow.reported.battery:{70 TO 100} // Sombra con valor del atributo “battery” en el rango (70, 100)
    - connectivity.connected:true // Dispositivos conectados
    - thingGroupName:lights // Grupo de cosas de nombre “lights”

Get me all the widgets prior to version 6 that support more than 3 colors

```
"attributes.version < 6 AND attributes.colors > 3"
```

Find all rooms where the temperature is above 20°

```
"shadow.reported.temperature>20 AND attributes.room_number:*"
```

F nd all my l ghtbulbs that are currently on and no one present n the warehouse

```
"shadow.reported.illumination_state = on AND shadow.reported.occupant present
 = no AND attributes.building_type = warehouse"
```

Find all connected devices

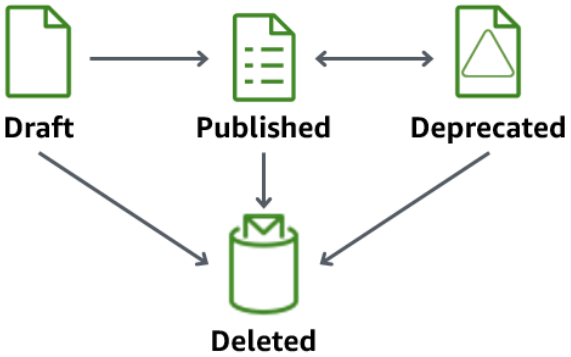
```
"connectivity.connected:true"
```



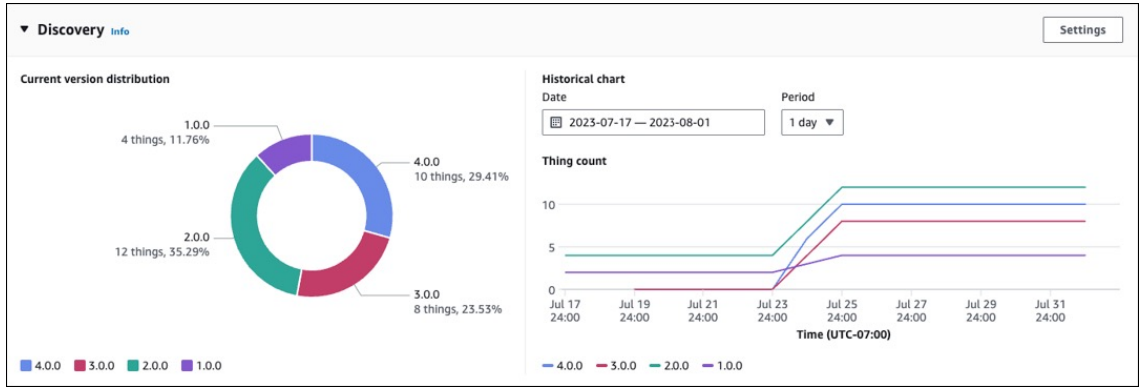
# Indexación de flotas



- Catálogo de paquetes software
  - Mantiene un inventario de paquetes software y sus versiones, que pueden asociarse a cosas individuales y grupos dinámicos de cosas y desplegarse usando trabajos de AWS IoT o procesos propios
    - Un paquete es una colección de archivos que se instalan como una única unidad
    - Puede contener *firmware*, actualizaciones del SO, aplicaciones, configuraciones y parches de seguridad
  - Pueden agruparse las cosas dinámicamente por los paquetes y versiones que tengan instalados y buscar que cosas tienen o no un paquete o versión instalada




Ciclo de vida de un paquete




Consola de AWS IoT mostrando el número de cosas y porcentajes que tienen una determinada versión de un paquete asociada

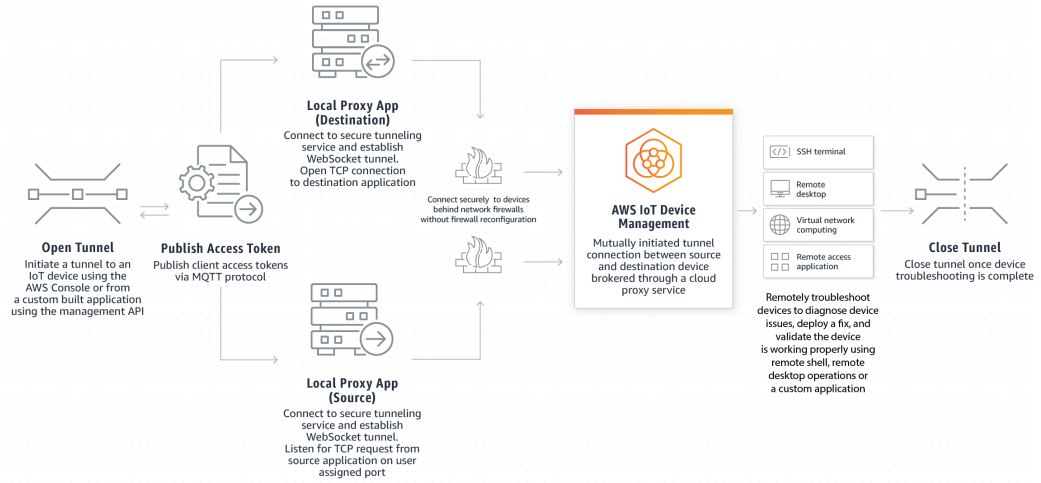


# Contenidos

- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - **Device Management**
    - Aprovisionamiento de dispositivos
    - Trabajos remotos
    - Indexación de flotas
    - **Tunelización segura**
  - Device Defender
  - Device Advisor
- Servicios de análisis de datos

# Tunelización segura

- ¿Qué es?
  - Solución para establecer una conexión segura gestionada por AWS IoT a dispositivos ubicados detrás de un *firewall* en lugares remotos
    - Para solucionar problemas, actualizar la configuración, etc
    - No requiere modificar las reglas del *firewall*
    - Reduce el tiempo de respuesta y recuperación ante incidentes y los costes operativos comparado p.ej. con el envío de un técnico
  - El origen y el destino se comunican usando un *proxy* local de código abierto que es ejecutado en cada extremo (p.ej. portátil del técnico y dispositivo remoto)
    - Los *proxys* se comunican con AWS IoT usando un puerto (p.ej. 443) abierto en el *firewall*
    - Los datos se transmiten cifrados usando TLS





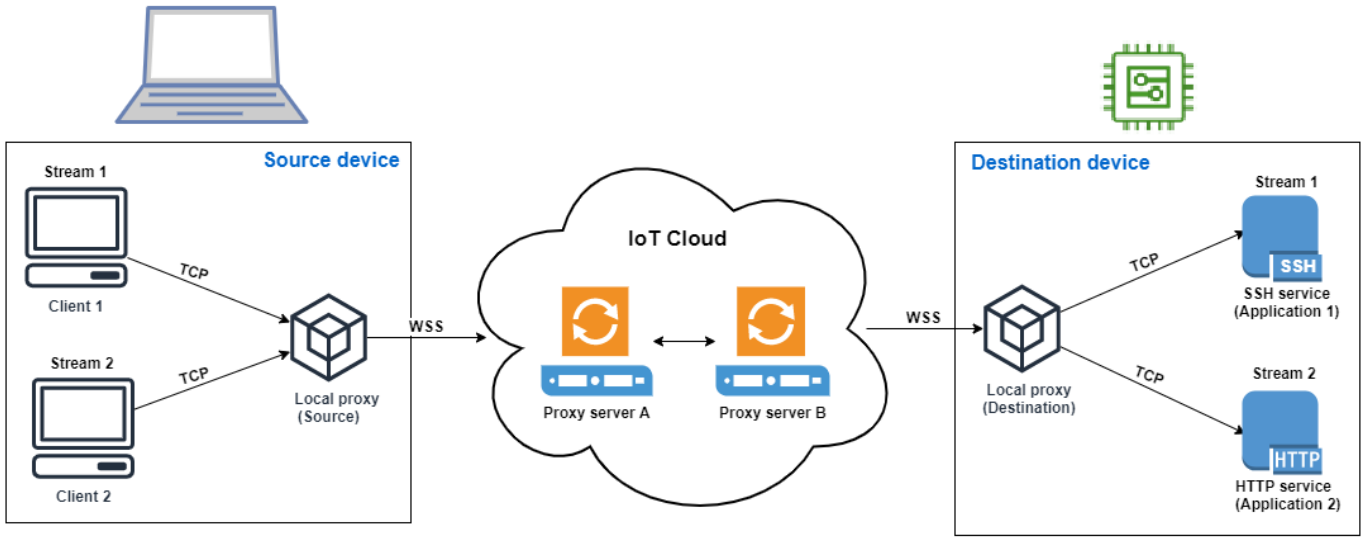
# Tunelización segura



- El agente de IoT
  - Aplicación que se ejecuta en el dispositivo remoto y que se conecta a AWS IoT para escuchar las notificaciones MQTT del túnel en el tema

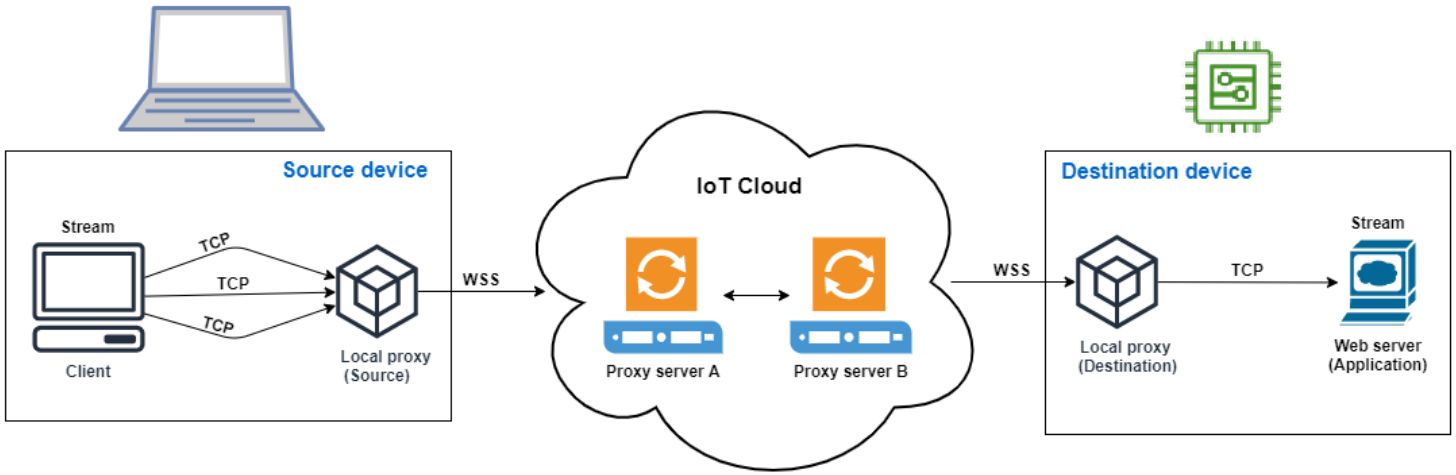
    `$aws/things/thing-name/tunnels/notify`
  - Se usa para recibir el mensaje MQTT que incluye el *token* de acceso cliente (CAT) del dispositivo e iniciar el *proxy* local en el dispositivo con los parámetros adecuados

- Multiplexación de flujos de datos
  - Permite compartir un túnel seguro entre varios flujos de datos
    - Ejemplo
      - Conexión HTTP a una aplicación web en el dispositivo para configurar parámetros de red
      - Conexión SSH a través del terminal para ejecutar comandos de *shell* para verificar que el dispositivo esté funcionando correctamente con los nuevos parámetros de red



# Tunelización segura

- Multiplexación de flujos de datos
  - Para cada flujo de datos, el túnel soporta varias conexiones TCP simultaneas
    - Ejemplo
      - Acelerar la descarga de un sitio web en el dispositivo remoto formado por varios componentes





# Contenidos

- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - Device Management
  - **Device Defender**
  - Device Advisor
- Servicios de análisis de datos

# Device Defender



- ¿Qué es?
  - Un servicio de seguridad y monitorización para auditar la configuración de los dispositivos, monitorizar dispositivos conectados y reducir riesgos de seguridad
    - Las flotas de dispositivos IoT son un objetivo atractivo para los hackers
      - Están formadas por un gran número de dispositivos distribuidos geográficamente, lo que complica su configuración haciendo que sea compleja y propensa a errores
      - Las capacidades de los dispositivos son reducidas, lo que limita el uso de cifrado y otras formas de seguridad
      - Los dispositivos suelen usar software con vulnerabilidades conocidas
    - Proporciona herramientas para auditar flotas de dispositivos comprobando si se adhieren a las mejores prácticas de seguridad y detectando comportamientos anómalos





# Device Defender



- Características




- **Auditoría**
  - Device Defender audita los recursos asociados a los dispositivos IoT para comprobar que cumplen con las mejores prácticas de seguridad
    - Dispositivos con el mismo certificado
    - Certificados caducados
    - Políticas demasiado permisivas que permitan a un dispositivo suscribirse a temas de otros, etc
  - La auditoría envía alertas si se detectan riesgos de seguridad y proporciona información relevante para ayudar a mitigarlos
  - Pueden hacerse auditorías bajo demanda o programarlas para que se hagan automáticamente

# Device Defender

- Características




DEPRECATED

- **Monitorización e identificación de anomalías**
  - Device Defender monitoriza las métricas de seguridad del dispositivo y del lado de la nube para identificar comportamientos anómalos y posibles dispositivos comprometidos, p.ej.
    - Cambios en los patrones de conexión, p.ej. un incremento provocado en los mensajes enviados a AWS IoT desde un dispositivo con la intención de subir los costes por mensajes
    - Dispositivos que se comunican con *endpoints* no autorizados o IP maliciosas
    - Cambios en los patrones de tráfico de E/S de los dispositivos, p.ej. picos en el tráfico de salida pueden indicar que el dispositivo está participando en un DDoS
  - Se envían alertas en el caso de detectar alguna anomalía
  - Dos métodos de identificación de anomalías
    - Usando **reglas definidas por el usuario** para especificar el comportamiento normal del dispositivo con el que comparar las métricas
    - Usando **modelos ML** de identificación de anomalías creados y mantenidos automáticamente
      - Los modelos se entrenan inicialmente durante 14 días usando hasta 6 métricas de la nube y 7 del dispositivo
      - Después se entrenan de nuevo diariamente



# Device Defender



- Características




- **Alertas y mitigación**
  - Los resultados de las auditorías programadas y de las anomalías detectadas están disponibles en la consola de AWS IoT y en CloudWatch
  - Pueden enviarse los resultados y alertas a Cloudwatch o a SNS para su integración en paneles de seguridad o el lanzamiento automático de acciones de mitigación como
    - Añadir cosas a un grupo de cosas
    - Sustituir la versión de la política predeterminada
    - Actualizar el certificado del dispositivo
    - Descargar parches de seguridad



# Contenidos

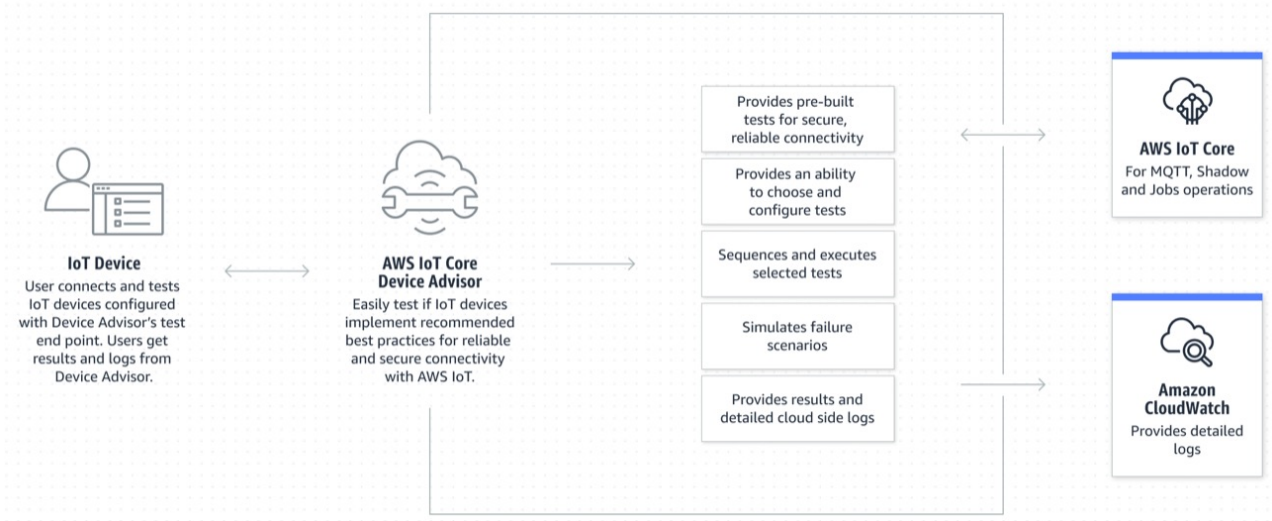
- Introducción
- Software de dispositivo
- **Servicios de control**
  - IoT Core
  - Device Management
  - Device Defender
  - **Device Advisor**
- Servicios de análisis de datos



# Device Advisor



- ¿Qué es?
  - Capacidad en la nube para la validación de la conexión segura y fiable de los dispositivos con IoT Core antes de ponerlos en el mercado
    - El dispositivo se configura para conectarse al *endpoint* de pruebas de Device Advisor
    - En IoT Core se configuran las pruebas a ejecutar escogiéndolas de un conjunto predefinido
    - Device Advisor gestiona el ciclo de vida completo de una prueba, incluyendo
      - El aprovisionamiento de recursos, la planificación de la prueba y la gestión de sus estados, el registro del comportamiento del dispositivo y de los resultados, y la elaboración del informe final de calificación
    - El informe final de calificación es válido para registrar el dispositivo en el catálogo de colaboradores de AWS *(AWS Partner Device Catalog)*





# Device Advisor



- Pruebas
  - Device Advisor proporciona seis categorías de pruebas predefinidas

**TLS**
- Comprueba si el protocolo de seguridad TLS entre los dispositivos y AWS IoT es seguro.

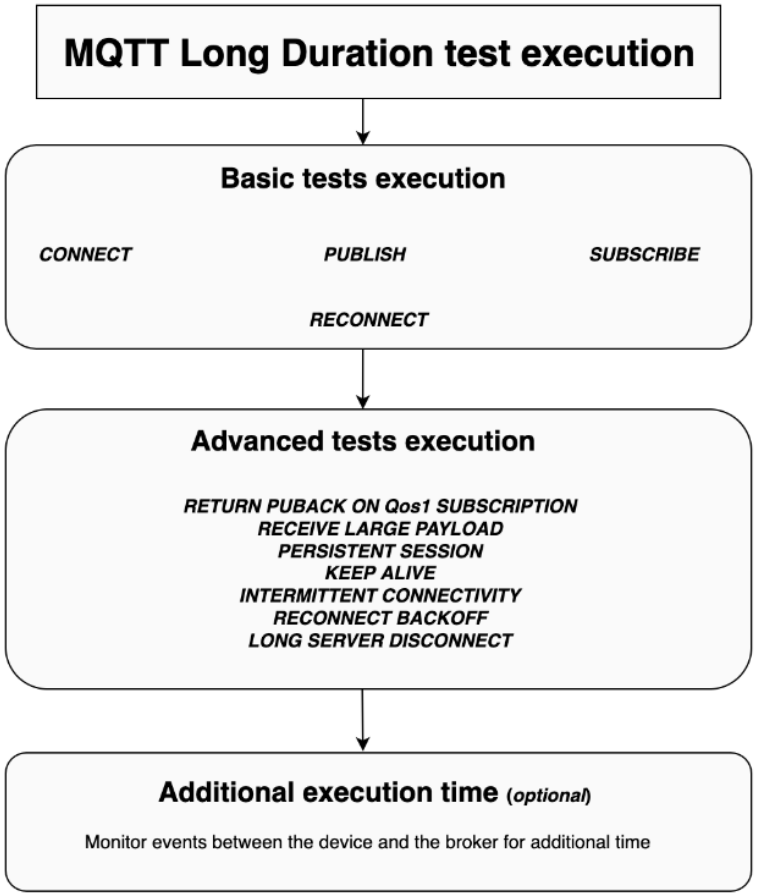
**MQTT**
- Comprueba si el dispositivo puede conectar, desconectar, reconectar, publicar, suscribir y como se comporta en otros escenarios más complejos.

**Sombra**
- Comprueba si los dispositivos usan correctamente las sombras AWS IoT.

**Ejecución de trabajos**
- Comprueba si el dispositivo puede recibir actualizaciones y publicar el estado de las actualizaciones exitosas usando AWS IoT.

**Permisos y políticas**
- Comprueba si las políticas asociadas a un dispositivo siguen las mejores prácticas y no conceden al dispositivo más permisos de los necesarios.

**Pruebas de larga duración**
- Pruebas que monitorizan el comportamiento del dispositivo cuando opera durante periodos de tiempo largos.





# Contenidos

- Introducción
- Software de dispositivo
- Servicios de control
- **Servicios de análisis de datos**



# Servicios de análisis de datos

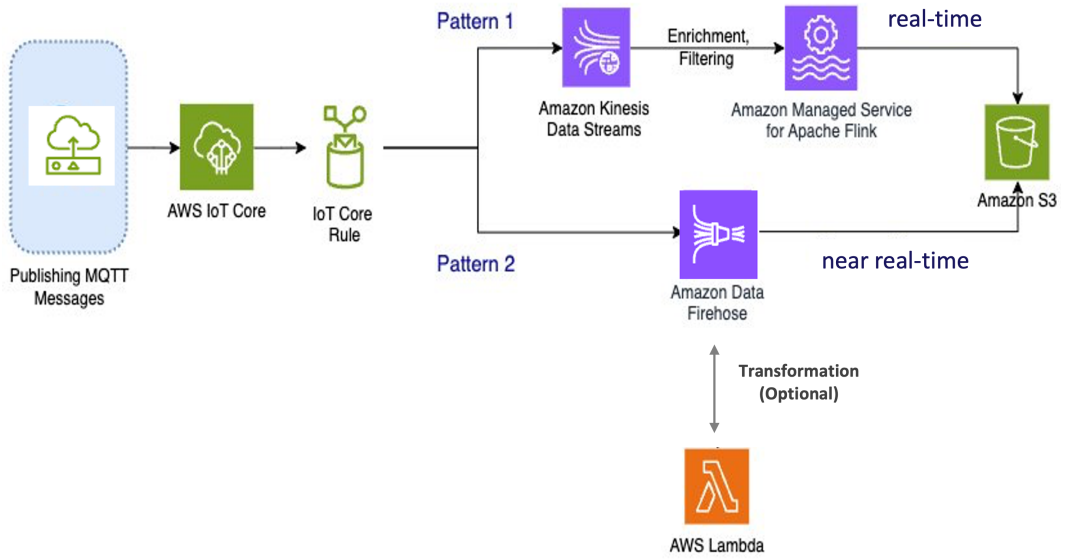


- Son los servicios para el análisis de datos IoT
  - **IoT Events** DEPRECATED
    - Servicio para detectar y responder a eventos de sensores y aplicaciones IoT
  - **IoT Fleetwise** DEPRECATED
    - Servicio para la colecta y transferencia de datos de vehículos a la nube casi en tiempo real (*near-real time*)
  - **IoT SiteWise** (+info)
    - Servicio para la recogida, organización y análisis de datos de equipos industriales a gran escala
  - **IoT TwinMaker** (+info)
    - Servicio para la creación de gemelos digitales

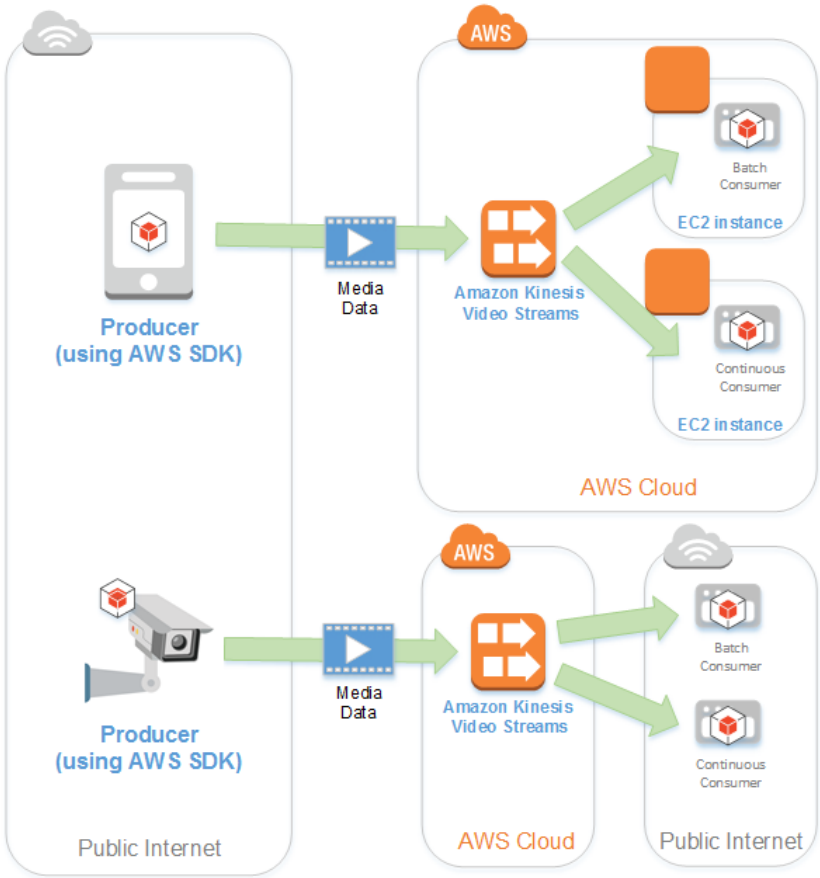


# Servicios de análisis de datos

- Streaming de datos
  - Patrones de arquitectura y servicios sugeridos por AWS
    - Patrón 1: Kinesis Data Streams con Amazon Managed Service for Apache Flink
      - Para aplicaciones de *streaming* de datos con requerimientos de gran ancho de banda y procesamiento avanzado
    - Patrón 2: Amazon Data Firehose
      - Para aplicaciones que solo necesiten procesamiento y transformaciones básicos
    - Ambos patrones almacenan los datos en S3 y recomiendan usar **Athena** para consultas SQL y **Glue** para procesamiento y transformaciones adicionales



- *Streaming* de vídeo
  - **Amazon Kinesis Video Streams**
    - Servicio gestionado para la transmisión en tiempo real de vídeo y su almacenamiento en AWS
    - Soporta todo tipo de fuentes: *smartphones*, cámaras de seguridad, *webcams*, cámaras en vehículos y drones, ...
    - También puede utilizarse para transmitir otros tipos de información: sonido, imágenes térmicas, datos de radar, etc
    - Proporciona APIs para crear aplicaciones de procesamiento de imágenes en tiempo real o su análisis *offline*





# Servicios de análisis de datos



- *Streaming desde Greengrass*
  - Desde dispositivos Greengrass, también se pueden transmitir datos y vídeo usando componentes proporcionados por AWS
    - *Stream manager*
      - Transmite grandes volúmenes de datos desde fuentes locales a AWS
    - *Firehose*
      - Transmite los datos a través de flujos de entrega de Amazon Data Firehose hacia destinos en AWS
    - *Edge connector for Kinesis Video Streams*
      - Transmite el vídeo capturado por cámaras locales a través de flujos de *Kinesis Video Streams* a AWS